

测试时训练的正确做法

Tianyuan Zhang¹ Sai Bi² Yicong Hong² Kai Zhang² Fujun Luan²
Songlin Yang¹ Kalyan Sunkavalli² William T. Freeman¹ Hao Tan²

¹Massachusetts Institute of Technology ²Adobe Research

摘要

测试时训练 (Test-Time Training, TTT) 通过在推理时调整模型部分权重 (通常称为快速权重) 来建模上下文依赖关系。这种调整后的快速权重类似于循环神经网络 (RNN) 中的循环状态, 存储当前序列中过去词元的临时记忆。现有的 TTT 方法由于在现代图形处理器 (GPU) 上计算效率低下, 难以证明其在处理长序列数据方面的有效性。许多这类方法中的 TTT 层浮点运算利用率极低 (通常低于 5%), 因为它们刻意采用较小的在线小批量大小 (例如每 16 或 64 个词元更新一次快速权重)。此外, 小批量意味着数据中存在细粒度的块级因果依赖关系, 这使得它们不适用于一维有序序列之外的数据, 例如集合或图像、视频等 N 维网格。相比之下, 我们反其道而行之, 提出了一种极大的块更新策略, 在不同模态的任务中块大小从 2K 到 1M 个词元不等, 我们称之为大块测试时训练 (Large Chunk Test-Time Training, LaCT)。该方法将硬件利用率提升了数个数量级, 更重要的是, 促进了非线性状态规模的扩展 (最高可达模型参数规模的 40%), 从而大幅提升了状态容量, 且无需繁琐且易出错的自定义核函数实现。它还便于集成诸如 Muon 之类的先进优化器用于在线记忆更新。我们在多种数据模态和任务上验证了该方法, 包括从图像集合进行新视角合成、语言模型以及自回归视频扩散模型。我们的方法可扩展至 140 亿参数的自回归视频扩散模型, 处理长达 56K 词元的序列。在我们最长的序列实验中, 我们以超过一百万的上下文长度进行了新视角合成。我们的结果凸显了大块测试时训练在计算和性能方面的优势, 为更高效、可扩展的长上下文序列建模铺平了道路。我们希望这项工作能够启发并加速长上下文建模和测试时训练领域的新研究。项目网站上有可视化结果 <https://tianyuanzhang.com/projects/ttt-done-right/>。

1 引言

处理长上下文的需求正在迅速增长。虽然 Softmax 注意力机制 (Softmax Attention) [1] 已成为建模各类数据的事实标准方案, 但其计算成本随序列长度呈二次方增长, 这促使了针对更高效长上下文建模的广泛研究。

近期, 测试时训练 (Test-Time Training, TTT) [2] 作为一种有前景的高效二次方序列建模方法崭露头角。TTT 将循环神经网络 (RNN) 中的循环状态概念扩展为一个小型的、在线自适应子网络。该子网络的参数也被称为快速权重 (Fast Weight) [3], 因为它们通过自监督目标在线快速自适应, 以记忆上下文信息。大量近期研究 [4, 5, 6, 7] 探索了快速权重网络的各种在线目标、优化器和架构。

尽管做出了这些努力，现有的 TTT 方法仍难以有效扩展到长上下文，主要原因是其 TTT 层的硬件利用率极低（在现代 GPU 上通常低于峰值 FLOPS 的 5%）。这种低效源于使用了小批量大小，即每个令牌或每 16 到 64 个令牌更新一次快速权重，这通常被认为对上下文学习更有效。如此小的批量导致并行性差和计算强度低，并对硬件高效实现提出了重大挑战，尤其是在使用大型非线性快速权重时，使得难以实现非平凡的（超过 10%）FLOPs 利用率。

本文采用相反的策略，提出了大块测试时训练（Large Chunk Test-Time Training, LaCT）。LaCT 利用极大的块（从 2048 到 100 万个令牌）作为更新快速权重的基本单元。由于每个大块内的令牌被视为无序集合，我们进一步将窗口注意力（Window Attention）集成到 LaCT 中，以捕获块内的局部依赖关系。LaCT 显著增强了并行性，从而大幅提高了 GPU 利用率（在 NVIDIA A100 上高达 70%），仅需几十行纯 PyTorch 代码（见附录 A.1）。这种效率使得非线性快速权重的扩展成为可能，以增强记忆容量。简单的实现允许轻松集成更有效的测试时优化器，例如 Muon[8]。

此外，LaCT 的大块设计也天然适合建模多样化的 N 维数据，因为我们可以将块大小与数据的内部结构对齐（例如，将图像内的标记或连续视频帧分组为一个块）。

我们在三个跨越不同模态和数据结构的任务上广泛验证了 LaCT：• 新视角合成。我们的模型能够以 960×536 的分辨率处理多达 128 张输入图像，从而产生最多 1M 个标记，并在这种输入规模下在渲染质量方面优于 3D 高斯泼溅 [9]。

- 语言建模。尽管语言数据中并未显式存在块结构，我们的模型仍取得了与 DeltaNet [10] 等最先进方法相比具有竞争力的性能。
- 自回归视频扩散。我们通过将 LaCT 与滑动窗口注意力相结合，将一个 140 亿参数的双向视频扩散 Transformer 适配为自回归模型。该适配模型能够生成多达 56,000 个视觉标记的一致视频。

总而言之，我们的方法为跨多种模态的长序列建模建立了一个高效、可扩展且高性能的框架。通过消除对底层硬件特定实现的依赖，LaCT 使得对架构设计空间的更广泛探索成为可能。我们相信这能够使高效长上下文建模的研究更加普及，并激发更多新颖且有效设计的开发。

2 预备知识

2.1 测试时训练

考虑一个由 N 个标记组成的一维序列 $\mathbf{x} = [x_1, x_2, \dots, x_N]$ ，其中每个标记 $x_i \in \mathbb{R}^d$ 。遵循注意力公式，每个输入标记 x_i 被投影为查询（ q_i ）、键（ k_i ）和值（ v_i ）向量。为清晰起见，我们假设所有这些向量 $q_i, k_i, v_i \in \mathbb{R}^a$ 。

测试时训练（Test-Time Training, TTT）[2] 引入了一种具有快速可调权重的神经网络——称为快速权重 [3]——在训练和推理过程中都会更新，以动态存储上下文信息。这与推理期间冻结的慢速权重（即模型参数）形成对比。形式上，TTT 以神经网络的形式定义快速权重：

$f_W(\cdot) : \mathbb{R}^d \rightarrow \mathbb{R}^d$ 由快速权重 W 参数

化，并涉及两个主要操作：更新操作：其中 $W \leftarrow W - \eta \nabla_W \mathcal{L}(f_W(k), v)$ (1)

$\mathcal{L}(\cdot, \cdot)$ 是变换后的键 $f_W(k)$ 与值 v 之间的损失函数，通常为均方误差，旨在鼓励网络将键与相应的值关联起来。 η 是学习率。直观上，该学习目标是使键值缓存编码为具有固定状态大小的神经记忆，并尽可能准确 [4]。应用操作：使用更新后的快速权重 W 根据查询 q 计算输出向量 o 。逐令牌的 TTT 层对序列中的每个令牌 x_i 依次 $o = f_W(q)$ ，执行更新和应用操作。 (2)

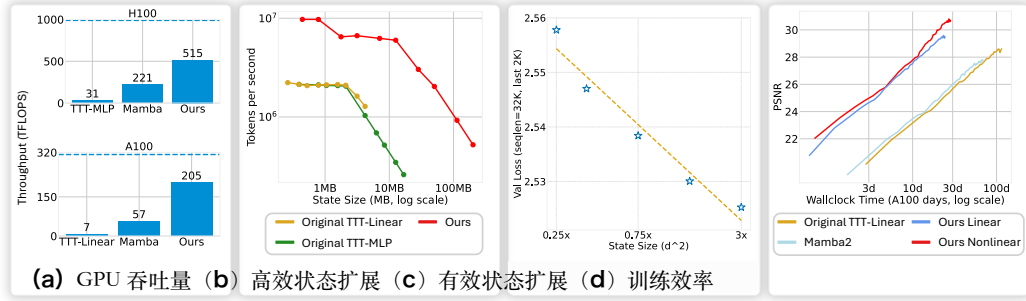


图 1：与原始测试时训练（TTT）方法相比，使用更大的块大小显著提高了 GPU 利用率，即使原始方法使用了定制核函数（a）。这种增强的利用率使得能够高效且有效地扩展到更大的状态大小（b）、（c），从而在更少的墙钟时间内获得更好的整体性能（d）。（a）中的虚线是 GPU 的理论峰值 BF16 吞吐量。面板（c）测量了 LaCT 语言模型在不同状态大小下处理的序列中最后 2K 个令牌的平均验证损失，展示了更大状态大小的优势。面板（d）在新视角合成基准上比较了不同基线的性能与训练时间。更多实验细节见第 C.4 节。

2.2 高效实现中的挑战

由于内存带宽限制，频繁在线更新快速权重效率低下。因此，先前的工作 [11, 12, 13, 14, 15] 通常采用定制核函数，在更新过程中将快速权重保留在静态随机存取存储器（SRAM）中，以减少内存负载。然而，这种策略通常要求快速权重在流式多处理器（SM）内基本独立演化，以减少通信，这对于大型非线性状态（例如第 3.1 节中的非线性 SwiGLU 快速权重和第 3.2 节中的 Muon 更新）并不适用。此外，开发此类核函数代码十分繁琐，开发周期远长于原生 PyTorch 代码，阻碍了快速的研究探索。

另一方面，基于 PyTorch 的实现虽然更简单，但通常受限于内存速度。举例来说，考虑一个简单多层感知机（MLP）快速权重的 PyTorch 实现，其核心是快速权重（例如 $h \times h$ 矩阵）与小批量输入（ $b \times h$ ，其中 b 为块大小）之间的矩阵乘法。理想的计算与内存比为：

$$r = \frac{2h^2b}{2h^2 + 4hb} = \frac{h/2}{1 + \frac{h}{2b}} = \frac{b}{1 + \frac{2b}{h}} \leq \min(h/2, b). \quad (3)$$

此处， $2h^2b$ 是矩阵乘法的浮点运算次数，分母 $2h^2 + 4hb$ 是两个输入矩阵和输出在 BF16（2 字节）下的内存工作量。较小的快速权重尺寸（例如 $h = 64$ ）或较小的块大小（例如 $b = 16$ ）将使该比值 r 远低于理论峰值（例如 H100 上每字节 290 次浮点运算），从而使该操作受限于内存，限制了计算利用率。

鉴于此，我们提倡使用较大的块大小（从 2048 到 1M）。这使我们能够实现更高的吞吐量（图 1a），从而在更少的训练墙钟时间内获得更好的性能（图 1d）。我们的设计还允许状态大小高效扩展（图 1b），从而通过这种扩展显著改善结果（图 1c，图 7a）。我们的架构实现了状态与参数大小之比 $\geq 40\%$ ，比先前方法的比值 0.1% 至 5% 高出一个数量级。详细伪代码见附录 1。

序列长度维度上的并行 除了批维度和头维度之外，序列维度的并行对于处理长序列（此时批大小通常较小）时实现高占用率至关重要。线性注意力变体如曼巴（Mamba）[12]、门控线性注意力（Gated Linear Attention）[13] 和德尔塔网络（DeltaNet）[15] 通过利用线性递归的结合性质实现了这种并行。注意力机制 [1, 16] 可以利用在线 Softmax 函数 [17] 沿序列长度维度进行并行化，这是闪电注意力 2（FlashAttention-2）[16] 相对于闪电注意力 1（FlashAttention-1）[18] 的关键改进。对于具有非线性更新的测试时训练，序列维度并行只能在在线块内实现，这进一步促使使用极大的块大小。在使用 PyTorch 实现大块测试时训练时，设备内跨多个线程块的这种序列维度并行由 PyTorch 和底层编译器自动处理。跨多个设备的这种序列并行的示例在第 3.4 节中提供，伪代码见附录 3。

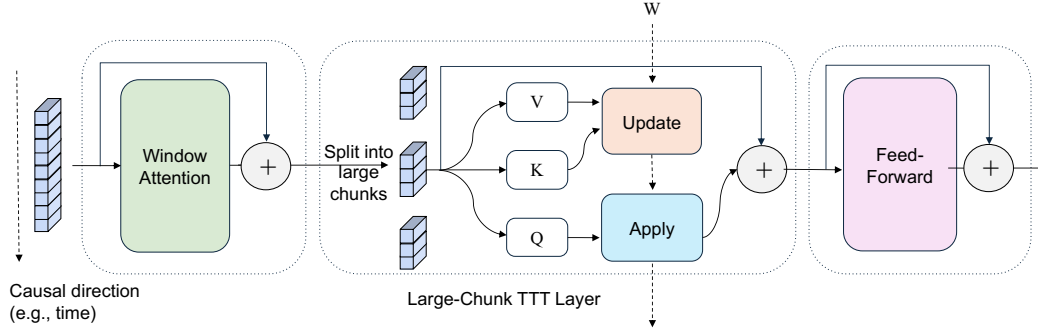


图 2: 拉康特 (LaCT) 块的基本示意图。大块测试时训练层更新快速权重 W 以存储历史上下文信息，而窗口注意力处理块内的局部性和内部结构。实线表示模型深度上的信息流，虚线表示时间上的信息流（即快速权重 W 在块间传递）。第 4 节中的各种实例化根据具体的数据结构使用不同的块大小和窗口注意力类型。此外，窗口注意力和大块测试时训练层可以通过共享查询键值 (QKV) 并对其输出求和而在同一层内结合；这种层内混合用于我们的语言建模和视频生成实验（此类伪代码见附录 2）。

3 拉康特 (LaCT) 模型架构

如图 2 所示，LaCT 块由三种类型的层组成：窗口注意力层、大块 TTT 层和前馈层。每层均配备残差连接 [19]，遵循变换器 (Transformer) [1] 中的实践。窗口注意力层执行局部自注意力以捕获局部依赖。在 TTT 层中，我们将序列分割成大块。历史上下文通过“更新”操作（关于键向量 K 和值 V ）逐渐压缩到快速权重中，最新权重被“应用”到当前查询向量 (Q) 以计算其对应输出。前馈层执行与变换器相同的通道混合。为清晰起见，我们在图 2 中省略了若干线性层和归一化层，细节见附录 A.1。我们的框架在处理多样数据类型时提供了极大的灵活性。在本节中，我们介绍方法中的通用设计，并在第 4 节描述数据特定的变体。

3.1 大块 TTT 层

与公式 1 中的逐令牌更新不同，逐块更新计算块内所有键 $\{k_i\}$ 和值 $\{v_i\}$ 的总损失梯度。由于块大小较大，权重更新不频繁。这使得更复杂的权重更新规则设计成为可能（在第 3.2 节讨论），并分摊了更新成本。快速权重的“更新”操作为：

$$g = \nabla_W \sum_{i=1}^b \eta_i \mathcal{L}(f_W(k_i), v_i) \quad (4)$$

$$W \leftarrow \text{weight-update}(W, g), \quad (5)$$

其中 b 是块大小， g 是快速权重损失函数的梯度， η_i 是每个令牌的学习率（通常由输入令牌预测）。“应用”操作 $o_i = f_W(q_i)$ 与公式 2 相同，块中的所有查询向量 $\{q_i\}$ 共享相同的更新后快速权重 W 。

受近期大型语言模型 (LLMs) [20] 的启发，我们采用无偏差项的 SwiGLU-MLP[21] 作为快速权重网络。我们的快速权重由三个权重矩阵 $W = \{W_1, W_2, W_3\}$ 组成，网络为：

$$f_W(x) = W_2 [\text{SiLU}(W_1 x) \circ (W_3 x)] \quad (6)$$

其中 $\circ$ 是逐元素乘法。我们采用简单的点积损失作为损失函数：

$$\mathcal{L}(f_W(k_i), v_i) = -f_W(k_i)^\top v_i \quad (7)$$

“应用”和“更新”的执行顺序。注意，TTT 的“更新”操作和“应用”操作是解耦的，我们可以自适应地设置块大小，并在不同的

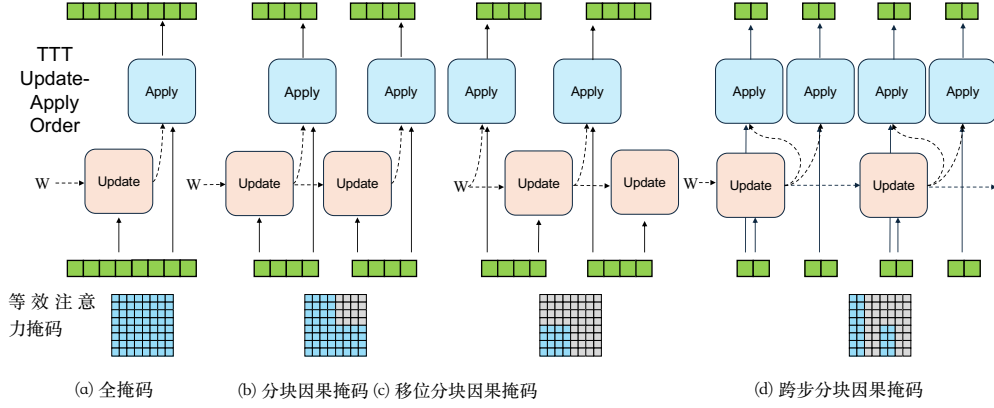


图 3: 不同的“更新”与“应用”顺序及其等效注意力掩码。第 i 行第 j 列的蓝色掩码表示第 i 个标记的输出依赖于第 j 个标记。

顺序；这使我们能够建模多种数据依赖关系，类似于自注意力中不同的注意力掩码。图 3 展示了这一概念。在图 3a 中，当块大小等于整个序列长度时，先执行应用操作再执行更新操作在概念上类似于全注意力。交替使用更新和应用操作会产生分块因果掩码（图 3b），其中块大小对应于块大小。交换两种操作的顺序会导致掩码发生移位（图 3c）。这种移位掩码不会在块内泄露未来信息，并且在语言建模中构建完整因果掩码时非常重要（第 4.2 节）。此外，仅对部分块进行更新而对所有块进行应用（图 3d）类似于跨步分块因果掩码。

3.2 快速权重的非线性更新

TTT 中的快速权重更新会反复累积梯度，因此会遭受幅度爆炸或记忆衰减的问题。大块 TTT 允许非线性更新，以提高稳定性和有效性，同时保持效率。对于公式 5 中的“权重更新”操作，我们的基础实现包括梯度下降后进行权重归一化：权重更新 $(W, g) = \text{L2-归一化}(W - g)$ 。

(8)

我们还探索了一种更稳健的非线性 Muon [8] 更新规则 ¹，并进行了权重归一化：

$$\text{weight-update}(W, g) = \text{L2-Normalize}(W - \text{Muon}(g)) \quad (9)$$

快速权重归一化。我们对更新后的快速权重沿输入维度应用 L2 权重归一化 [22]。与先前方法 [5, 23, 13, 11] 不同，我们不使用显式的权重衰减项。当网络在概念上旋转 90 度，将序列维度视为虚拟模型的深度时，测试时训练更新在时间上充当残差 [19]。在这种视角下，我们的快速权重归一化类似于变换器（Transformer）架构中的后层归一化，它约束了残差路径内的激活尺度。

Muon 更新规则。本质上，Muon 通过牛顿-舒尔茨迭代对矩阵梯度的谱范数进行归一化。简而言之，设 $g = USV^T$ 为梯度 g 的奇异值分解（Singular Value Decomposition, SVD），则 Muon 算子近似地将梯度转换为：

$$\text{Muon}(g) \simeq UV^T \quad (10)$$

Muon 还提高了我们设置中的数值稳定性。例如，学习率（公式 4 中的 η_i ）现在仅反映块内标记的相对重要性，因为 Muon 对绝对尺度进行了归一化。有关其计算成本的分析，请参见 [8] 和附录 A。

3.3 窗口注意力

大块 TTT 层将数据视为集合的序列，因为其快速权重更新本质上忽略了每个块内的标记顺序和空间局部性。然而，许多数据模态——例如 1Muon 要求权重为矩阵形式，而我们当前的快速权重函数 SwiGLU-MLP 有三个矩阵作为权重（即公式 6 中的 W_1, W_2, W_3 ）。

如视频（网格序列）、图像集合（网格集）或文本（一维序列）——并不完全符合这种基于集合的视角。对于这些模态，块内结构和局部性对于捕捉整体数据结构至关重要。因此，我们在 TTT 层旁边集成局部窗口注意力（因果或双向）来处理块内的数据结构。此外，窗口注意力高效地处理数据中的局部性，使 TTT 层能够将其固定大小的快速权重容量专注于建模非局部依赖。这种混合策略也被其他著名工作采用，如 BASED [24]、GAU [25] 和 InifinitAttention [26]。总之，LaCT 是一种混合架构，使用二次计算的注意力处理局部结构，使用线性计算的 TTT 处理非局部上下文。

3.4 上下文并行

上下文并行（Context Parallelism, CP）沿上下文长度维度划分序列，并将分片分布到多个设备上并行计算。前馈层和窗口注意力是局部算子，因此原生支持 CP。对于 TTT 层，小块很难支持 CP，因此更倾向于使用张量并行（即按头并行）。我们的大块 TTT 层允许通过分片块内的令牌来实现 CP。假设每个分片包含 s 个令牌，由于梯度的线性性质，块的快速权重梯度是所有分片梯度之和：分片 分片

$$g = \nabla_W \sum_{j=1}^s \sum_{i=1}^s \eta_i \mathcal{L}_i = \sum_{j=1}^s \nabla_W \sum_{i=1}^s \eta_i \mathcal{L}_i \quad (11)$$

这可以通过分布式全归约求和来实现，逻辑上与分布式数据并行（Distributed Data Parallelism, DDP）相同，只是参数是快速权重，输入数据是块中的令牌。我们在训练新视角合成任务时采用了这种并行方式（见第 4.1 节），并观察到最小的吞吐量开销（1% 到 3%）。LaCT 架构与其他并行策略（如数据并行、流水线并行和张量并行）兼容。有关为 LaCT 实现上下文并行（算法 3）和张量并行（算法 4）的伪代码，请参见附录。

4 用于 N 维数据的 LaCT

在本节中，我们介绍使用 LaCT 解决的三个任务——新视角合成、语言建模和自回归视频生成。这些任务具有不同的固有数据结构，我们通过相应的设计选择来解决它们。这些数据类型的完整模型架构细节在附录 B 中提供。

4.1 新视角合成 - 图像集

新视角合成（Novel View Synthesis, NVS）[27, 28] 旨在从先前未见的视点渲染静态场景的图像。形式上，给定一组 N 输入位姿图像 $\{(I_i, P_i)\}_{i=1}^N$ ，其中 $I_i \in \mathbb{R}^{H \times W \times 3}$ 为 RGB 图像， P_i 为其对应的相机位姿，模型需要从通常与输入视图不重叠的新相机位姿合成新图像。

我们发现 NVS 是评估模型在线记忆与压缩能力的有效测试平台。首先，NVS 具有挑战性，因为它需要空间压缩、稠密检索和基本的物理推理。其次，NVS 可以表述为非生成式任务，显著降低训练计算量和对大量模型参数存储世界知识的需求，从而支持快速实验。第三，稠密输入视图中存在的大量冗余信息促使模型学习有效的压缩。基于这些观察，我们在初步研究迭代中采用 NVS。我们发现所获得的一些洞见可迁移至其他任务。

我们的 NVS 模型遵循第 3 节中的基本 LaCT 图示。位姿输入图像和目标新视图的位姿均通过分块和线性层进行标记化，遵循 LVSM[29]。窗口注意力恰好覆盖来自单张图像的标记。LaCT 层采用单轮分块步进因果掩码（图 3d），利用所有输入图像标记更新快速权重，并同时应用于输入和目标标记。更新步骤类似于预填充阶段，而应用操作类似于并行解码。在渲染新视图期间，每个测试时训练层充当静态权重层，使整个模型成为静态视觉变换器 [30]。我们在图 9 中展示了这一设计。

表 1: 我们在三种不同数据结构上的实验总结。‘d’表示模型维度。状态大小表示每个模型块的快速权重大小。

Task name	Data Structure	Chunk Size	State Size	Model Size	Max Length	Context Parallelism
Novel View Synthesis	Image set	Full sequence	$6d^2$	0.3B	1M	Within-chunk parallel
AR Video Diffusion	Image sequence	Three frames	$3d^2, 0.75d^2$	1.3B, 14B	56160	Head-dim parallel
Language Models	1D Sequence	2K, 4K tokens	$0.75d^2$	0.7B, 3B	32768	N/A

4.2 语言建模——文本序列

自回归语言模型根据前文词元预测下一个词元的概率分布， $p_\theta(x_n|x_1, \dots, x_{n-1})$ 。文本序列缺乏固有的块结构，因此对于 LaCT，我们将块大小定义为超参数（例如 2048 或 4096 个词元）。我们采用图 3(c) 中所示的移位分块因果掩码用于 TTT 的更新-应用序列，以避免在块中看到未来词元。由于 LaCT 在每个块内缺乏逐词元因果性，我们采用滑动窗口注意力——窗口大小等于块大小——以高效地建模逐词元因果依赖关系。滑动窗口被集成到同一个 TTT 层中，共享 QKV，类似于 GAU[25]。我们在图 10 和伪代码 2 中展示了详细架构。

4.3 自回归视频扩散——图像序列

分块自回归视频扩散迭代地去噪若干后续视频帧，条件于先前生成的干净帧，其中每个块可以包含数千个视觉词元。我们通过交错噪声块和干净帧块使用教师强制训练。具体来说，一个包含 N 个帧块的视频被结构化为：

$$S = [X_1^{\text{noise}}, X_1, X_2^{\text{noise}}, X_2, \dots, X_N^{\text{noise}}] \quad (12)$$

其中每个噪声块 X_i^{noise} 是通过向第 i 个干净视频块添加单位高斯噪声 ϵ 产生的，即 $X_i^{\text{noise}} = X_i(1 - t_i) + \epsilon t_i$ ，并且 $t_i \in [0, 1]$ 表示与块无关的噪声强度。

为了处理这种数据结构，我们采用图 3d 中用于 LaCT 的跨步分块因果掩码。具体来说，它按顺序对每个块应用快速权重，同时仅在干净块上更新快速权重。这种简单策略确保每次去噪操作仅访问先前已清理的帧。窗口注意力使用包含 2 个连续块 (i.e., $[X_i, X_{i+1}^{\text{noise}}]$) 的非重叠窗口来构建时间和空间局部性。在每个窗口内，从 X_i 到 X_{i+1}^{noise} 的注意力被排除。我们通过将所有注意力和 TTT 掩码模式移位（类似于图 3c）来纳入第一个噪声块。这种混合架构的细节和更高效的训练方法见附录 B.3。

5 实验

在本节中，我们展示了在新视角合成（第 5.1 节）、语言建模（第 5.2 节）和自回归视频生成（第 5.3 节）上的实验结果，以及对不同设计选择的深入分析（第 5.4 节）。表 1 总结了每个实验中的关键因素。在与线性成本基线进行比较时，我们为它们增加了相同的窗口注意力以进行公平比较。所有任务的完整实验细节见附录 C。

5.1 新视角合成

数据集与度量。 我们在物体级和场景级数据集上评估所提方法。物体级训练采用 Objaverse 数据集 [31]，遵循 LVSM[29] 和 GS-LRM[32] 的设置。训练完成后，在 Google Scanned Objects (GSO) 数据集 [33] 上进行评估，分辨率为 256×256 和 512×512 。每次评估包含每个物体 4 - 48 个输入视角和 8 个新视角。对于场景级评估，采用具有挑战性的 DL3DV 场景数据集 [34]，包含超过 11K 个训练场景和 140 个测试场景，每个场景约 300 个视角。评估分辨率为 960×536 。性能通过新视角的峰值信噪比 (PSNR) 衡量，其他度量见附录 C.1。

模型细节。 模型的每个块包含一个逐图像窗口注意力层、一个 SwiGLU-MLP 大块 TTT 层和一个前馈层。默认模型总参数量为 312M，其中快速权重 84M（每块 $6d^2$ ）。

表 2: 新视角合成方法的复杂度 (输入 n)。预填充和渲染速度在 A100 上测量, 输入为 48 张 512×512 图像 (196K 输入标记, 4K 解码标记)。

	状态大小	预填充计算量	解码计算量	参数量	预填充速度	渲染帧率
Full attention	$O(n)$	$O(n^2)$	$O(n)$	284M	16.1 s	2.3 FPS
Perceiver Attention	$O(1)$	$O(n^2)$	$O(1)$	287M	16.8 s	34.4 FPS
Ours	$O(1)$	$O(n)$	$O(1)$	312M	1.4 s	38.7 FPS

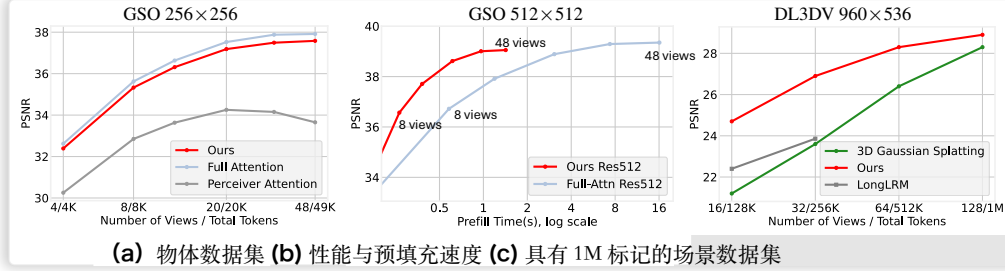


图 4: (a, b) 本文方法在显著降低预填充延迟的同时, 达到了与全注意力模型相当的质量, 并明显优于感知器注意力基线。(c) 在高分辨率场景数据集上, 本文方法超越了仅支持 32 个视图的 LongLRM, 并在稀疏视图下优于 3D 高斯泼溅, 在多达 128 个输入视图 (总计 1M 个标记) 时仍保持竞争力。

基线。对于物体级评估, 本文使用两种基线: 全注意力模型和感知器风格的寄存器注意力模型 [35]。全注意力基线将 TTT 层替换为块级因果注意力层, 实现输入标记之间的双向交互以及来自新视图的交叉注意力。感知器风格基线将输入标记压缩为 4096 个寄存器, 通过交叉注意力解码新视图。对于场景级评估, 本文与 LongLRM [36] (一种结合 Mamba [12] 和全注意力进行 3D 高斯泼溅预测的最先进模型) 以及纯基于优化的 3D 高斯泼溅方法进行比较。表 2 总结了所有模型的计算复杂度。

训练细节。对于物体数据集, 本文使用 1.25 万亿标记以渐进分辨率训练所有模型。对于场景数据集, 本文使用 1.8 万亿标记以渐进更高的分辨率和更多视图训练模型, 最大序列长度为 100 万标记。高分辨率模型使用块内上下文并行 (第 3.4 节) 进行训练。更多细节见第 C.1 节。

结果。实验结果与分析如图 4 所示。

5.2 语言建模

数据集与度量。我们在长数据集集合数据集上训练模型 [37], 使用其总计 688 亿个词元中的约 600 亿个词元。对于评估, 我们采用来自 [38] 的逐词元损失度量, 评估模型有效利用完整上下文的能力。单调递减的损失表明成功的上下文利用, 而趋于平稳则表明有限的上下文使用。此外, 我们报告了在不同序列长度下的检索准确率 [39]。

模型细节。我们从原始的拉切特块中移除了窗口注意力层, 将滑动窗口注意力层直接集成到大块测试时训练层中。遵循门控注意力单元 [25], 滑动窗口注意力与快速权重网络共享查询、键和值向量, 并对查询和键进行额外的逐通道缩放和偏移。该设计的伪代码见算法 2。

基线。我们与全注意力、门控线性注意力 [13]、德尔塔网络 [3, 15] 进行比较。为确保公平性, 我们为门控线性注意力和德尔塔网络都增强了相同的滑动窗口注意力。基于先前工作 [38, 40, 41] 强调了大旋转位置编码 [42] 基数对于长上下文变换器训练的重要性, 我们采用 100 万的旋转位置编码基数进行 32K 词元上下文的训练。表 3 总结了所有方法的机制和训练吞吐量。

训练细节。我们使用 32768 个词元的序列长度训练了两种规模的模型: 一个 7.6 亿参数的模型训练了 400 亿个词元, 滑动窗口为 2048 个词元; 一个 30 亿参数的模型训练了 600 亿个词元, 滑动窗口为 4096 个词元。更多细节见第 C.2 节。

表 3: 基线方法在状态大小、训练吞吐量（以每秒词元数衡量）、更新规则和内存读出机制方面的比较。训练吞吐量使用 30 亿参数模型在 A100-40GB 图形处理器上以 32K 序列长度进行评估。

	State size	Train TPS	Update Rule	Memory read-out
Transformer	—	4.1K	—	—
Transformer SWA	—	6.4K	—	—
<i>Per-token recurrence</i>				
GLA SWA	384d	5.0K	$\mathbf{S}_t \leftarrow \mathbf{S}_{t-1} \text{Diag}(\boldsymbol{\alpha}_t) + \mathbf{v}_t \mathbf{k}_t^\top$	$\mathbf{o}_t = \mathbf{S}_t \mathbf{q}_t$
DeltaNet SWA	128d	5.1K	$\mathbf{S}_t \leftarrow \mathbf{S}_{t-1} (\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top) + \beta_t \mathbf{v}_t \mathbf{k}_t^\top$	$\mathbf{o}_t = \mathbf{S}_t \mathbf{q}_t$
<i>Large-chunk recurrence</i>				
Ours GD	2304d	5.0K	$W \leftarrow \text{L2norm}(W - \sum_i \eta_i \nabla_W \mathcal{L}_i)$	$\mathbf{o}_t = f_W(\mathbf{q}_t)$
Ours Momentum	2304d	4.9K	$M \leftarrow \beta M + \sum_i \eta_i \nabla_W \mathcal{L}_i; W \leftarrow \text{L2norm}(W - M)$	$\mathbf{o}_t = f_W(\mathbf{q}_t)$
Ours Muon	2304d	4.3K	$M \leftarrow \beta M + \sum_i \eta_i \nabla_W \mathcal{L}_i; W \leftarrow \text{L2norm}(W - \text{Muon}(M))$	$\mathbf{o}_t = f_W(\mathbf{q}_t)$

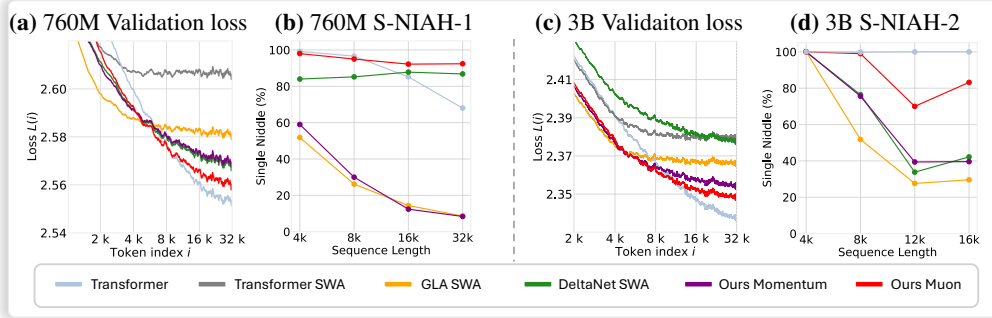


图 5: 语言模型结果。(a, c) 与门控线性注意力和德尔塔网络相比, 我们的模型在 7.6 亿和 30 亿规模下, 在较大的词元索引处实现了更低的逐位置损失, 表明更强的长上下文建模能力。(b, d) 我们的模型在检索准确率上始终优于门控线性注意力和德尔塔网络。此外, 我们的 μ 子变体始终优于我们的动量变体。

结果。请参考图 5 获取实验结果和分析。更多结果见第 C.2 节。

5.3 自回归视频扩散

我们将预训练的万 2.1[43] 文本到视频扩散模型微调为自回归视频扩散模型。具体而言, 我们将所有双向注意力层替换为我们的拉普拉斯变换层 (LaCT) 与滑动窗口注意力相结合。滑动窗口注意力使用的窗口大小跨越两个自回归块。

数据集。 我们使用内部经过筛选的专有视频集合对模型进行微调, 每个视频都配有由视觉语言模型生成的简短文本提示。

训练细节。 遵循 [44, 43], 我们采用时间步偏移和基于对数正态分布的降噪损失加权。我们在 16 帧每秒、480 × 832 分辨率下对 5 秒视频进行训练, 以 3 个潜在帧块进行自回归降噪。随后, 我们对 13 亿参数模型使用 10 秒视频进行微调, 对 140 亿参数模型使用 8.8 秒视频进行微调。每个 8.8 秒片段包含 56,160 个视觉标记, 在教师强制训练下产生交错的有噪-干净块, 总计 107K 个标记。我们对多层感知机层使用序列并行, 对测试时训练 (TTT) 和窗口注意力层使用张量并行 (跨设备分片头)。完整细节列于第 C.3 节。

基线。 我们将我们的方法与三个基线进行比较: 仅滑动窗口注意力 (SWA)、曼巴 2[23] 与 SWA 结合 (采用与我们的方法类似的并行组合策略), 以及全块状因果注意力。

评估。 我们在 5,000 次训练迭代后, 对包含 2,000 个视频的集合评估所有模型, 通过在五个时间步 (550、650、750、850、950) 计算降噪损失。图 6 绘制了

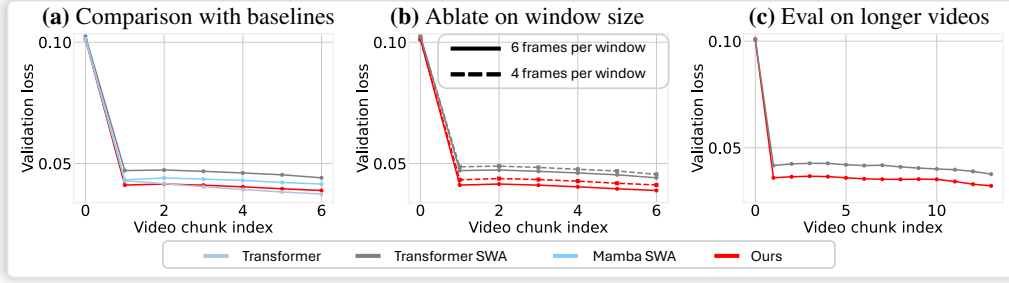


图 6: (a) 本文达到了与全注意力基线相当的验证损失，并优于采用滑动窗口的曼巴（Mamba）和滑动窗口注意力基线。这种相对于滑动窗口注意力的改进在不同窗口大小下保持一致（b），并且在更长视频上评估时也成立（c）。

分块去噪损失在评估的视频帧上计算。本文仅测量训练序列长度以内的验证损失。自回归生成的视频见项目网站²。

5.4 设计选择分析

本节分析模型中若干关键设计选择，重点关注新视角合成和语言建模任务，因为这些任务存在良好的度量指标。具体而言，本文评估状态大小（图 7a）、测试时优化器（图 7b）、线性与非线性快速权重（图 8a）以及逐令牌递归与分块递归（图 8b）的影响。总体而言，本文发现较大的状态大小、先进优化器（如 Muon）以及非线性快速权重能显著提升模型性能。在比较分块递归与逐令牌递归时，受控的新视角合成实验表明，在相同状态大小下，本文的线性大分块递归策略优于线性逐令牌递归。对于语言建模，分块结构并非固有，本文的线性大分块递归变体——虽然最初不如 GLA 和 DeltaNet 等逐令牌方法——但在结合大型非线性状态和 Muon 优化器后超越了它们。更详细的分析请参阅各图及其图注。

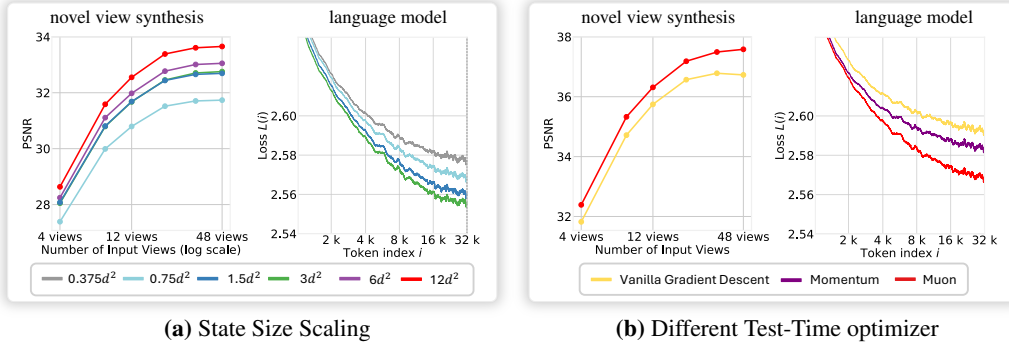
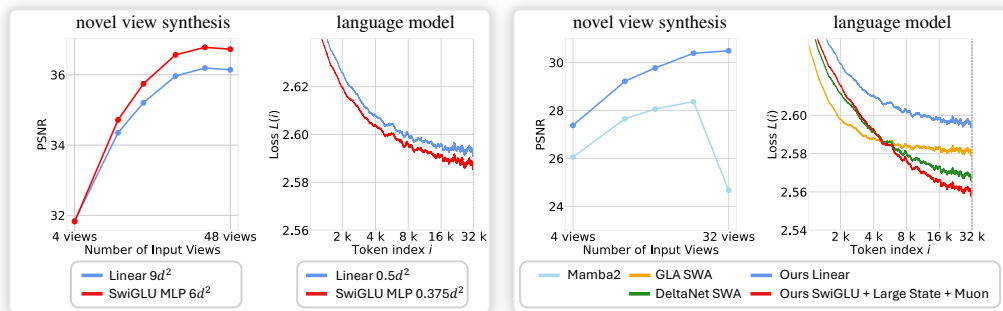


图 7: (a) 扩大状态大小在新视角合成和语言建模任务中均持续提升性能。注意，最大版本每块的状态大小为 $12d^2$ ，快速权重占模型权重的 40%。(b) 测试时优化器的比较表明，Muon 相对于普通梯度下降和动量法具有惊人的有效性。

状态大小扩展。这些受控实验使用 SwiGLU 多层感知机作为快速权重，并使用 Muon 作为测试时优化器。对于新视角合成，实验在物体数据集上进行。所有模型训练了 1670 亿个令牌，使用 14 个堆叠块，模型维度为 $d = 768$ 。为改变状态大小，本文保持头维度固定为模型维度，即单头，并改变 SwiGLU 多层感知机的中间维度，使中间维度范围从 192 到 3072。最大配置导致每个模型块的状态大小为 $12d^2$ ，总计

视频结果见项目网站: <https://tianyuanzhang.com/projects/ttt-done-right/>



(a) 线性与非线性快速权重 (b) 大块与逐令牌递归

图 8: (a) 非线性快速权重尽管使用更小的状态尺寸，仍始终优于线性快速权重。(b) 在相同状态尺寸下，我们的线性大块递归方法在视图合成任务中显著优于线性逐令牌递归（双向 Mamba2）。在语言任务中，相同状态尺寸的线性大块递归低于 GLA 基线，但当结合更大的非线性状态和 Muon 测试时优化器时，它超越了所有逐令牌递归方法。

模型权重的 40% 作为快速权重。对于语言模型实验，我们使用 7.6 亿参数设置，其中块大小和滑动窗口注意力（SWA）窗口大小设置为 2048 个令牌。我们保持快速权重 SwiGLU MLP 的中间维度与头维度相同。我们增加状态尺寸，同时按比例减少头数，以保持固定的模型维度。图 7(a) 表明，更大的状态尺寸始终提升性能。值得注意的是，小状态尺寸与大状态尺寸之间的性能差距随着序列长度的增加而扩大。

测试时优化器比较。我们将 Muon 与普通梯度下降（GD）和带动量的 GD 进行比较。动量实现的细节见附录 A。对于 NVS，我们使用 24 个堆叠块、模型维度为 768 的模型规格，对所有比较方法训练 671 个令牌。语言建模实验使用 7.6 亿参数设置。图 7(b) 显示 Muon 始终优于其他优化器。

线性与非线性快速权重。我们的默认快速权重函数是无偏置项的 SwiGLU MLP（非线性）。我们将其与简单的线性快速权重 $f_W(x) = Wx$ 进行比较。两者都使用相同的在线点积损失进行键值关联更新。图 8(a) 展示了 NVS 和语言建模的这一比较。尽管线性快速权重配置了比非线性 SwiGLU 更大的状态尺寸，但它们的性能较低。NVS 模型使用 24 个块和 $d = 768$ 训练了 671B 个令牌。语言建模使用 7.6 亿参数设置。

大块与逐令牌循环的对比。图 8(b) 展示了受控实验，比较了我们的大块循环与逐令牌循环。在新视角合成（NVS）任务中，“我们的线性”变体采用线性快速权重： $f_W(x) = Wx$ ，并与具有相同状态大小的 Mamba-2 基线（一种线性逐令牌循环模型）进行基准对比。为了适应 NVS 对输入图像令牌所需的双向上下文，Mamba-2 基线在每个模型块内使用两个方向相反的 Mamba-2 层。我们的线性变体和这种双向 Mamba-2 在每个块的状态大小均为 d^2 。这两种方法在每个模型块内都采用了每图像窗口注意力。在这种公平比较下，我们的线性大块循环实现了显著更好的视角合成性能。

对于图 8(b) 中同样展示的语言建模实验，蓝线“我们的线性”变体使用与 GLA SWA 基线相同的状态大小（ $0.25d^2$ ）。它最初表现不如 GLA SWA（蓝线低于黄线），这可能是因为语言数据缺乏有利于我们基本线性块循环的固有块结构。然而，当 LaCT 配备更大的非线性状态（ $1.5d^2$ ）和 Muon 更新时，我们显著优于这些逐令牌循环基线。

6 相关工作

测试时训练。测试时训练 (Test-Time Training, TTT) [2] 是序列建模中的一个新兴概念, 它将循环神经网络 (RNN) 中的循环状态概念扩展到在线适应的神经网络组件。在 TTT 模型中, 一部分权重 (称为“快速权重”) 被更新以进行上下文学习。现有方法通常采用自监督损失, 鼓励这些快速权重记忆来自上下文标记的键值关联, 并使用梯度下降的变体进行在线适应。TTT [2, 4] 为新的循环模型架构开辟了广阔的设计空间。例如, 许多近期工作开发了新颖的测试时优化器 [5, 7] 和在线训练目标 [45]。然而, 当前的 TTT 方法通常存在硬件利用率低和状态大小有限的问题, 因此尚未充分展示其潜力。我们的工作主要通过倡导一种使用极大在线小批量 (块) 大小来更新快速权重的新范式来解决这些挑战。该范式可以实现数量级更高的硬件利用率, 而无需依赖容易出错的自定义核函数实现。此外, 它能够高效扩展非线性状态大小, 并提供使用多样化快速权重神经网络和优化器的灵活性, 从而加速该领域的研究进展。

将块注意力与循环相结合。近期一些模型将局部块注意力与线性循环相结合, 例如门控注意力单元 (Gated Attention Unit, GAU) [46]、MEGA [47]、MEGALODON [48] 和 InfiniAttention [26]。其中, InfiniAttention 在概念上最接近我们的工作, 因为它使用增量规则在块级别引入循环——从测试时训练 (TTT) 的角度可解释为在线线性回归目标。然而, 这种更新规则的表达能力有限。相比之下, 我们采用了一种从更通用的 TTT 框架中推导出的、表达能力显著更强的更新机制, 并展示了由此带来的显著收益。

块递归变换器 [49] 也探索了大规模块记忆更新, 其中记忆令牌充当递归状态, 在每次块更新时通过注意力机制与输入令牌进行自注意力和交叉注意力。在我们的新视角合成实验中使用的感知器式寄存器令牌注意力基线 (第 5.1 节, 表 2) 在概念上与块递归变换器相似, 都使用寄存器令牌进行上下文压缩。如图 4 所示, 我们的方法在速度和图像质量上均显著优于该方法, 且状态大小相当。

新视角合成。新视角合成 (NVS) 是计算机视觉、图形学和计算摄影交叉领域的一项长期任务, 要求算法从先前未观察到的视点渲染静态场景的图像。基于优化的方法, 如神经辐射场 (NeRF) [50] 和三维高斯泼溅 (3D Gaussian Splatting) [9], 已取得重大突破。这些方法通过可微体渲染优化一组参数化的图形基元 (即辐射场的显式或隐式表示), 以最小化输入图像上的重建损失。经过通常持续数十分钟的优化过程后, 这些方法能够以照片级真实感渲染新视角, 并且优化后的参数构成了输入场景的三维表示。

最近, 数据驱动的方法 [32, 29, 36, 51] 也显示出有前景的结果。这些方法可以直接渲染新视角, 或在给定输入图像的情况下预测三维表示。尽管在较简单的物体数据集上取得了成功, 但这些方法在处理密集采样的场景 (例如, 输入图像超过 100 张的场景) 时往往表现不佳。我们的实验表明, 在具有挑战性的场景数据集上, 我们的分块测试时训练方法在最多 128 张输入图像、分辨率为 960×536 的条件下, 性能优于或与三维高斯泼溅相当。我们希望我们的方法能激发进一步的研究, 将数据驱动的新视角合成方法有效扩展到更长、更复杂的输入序列。

自回归视频扩散。当前最先进的视频生成主要由在潜空间中运行的双向扩散 Transformer 主导 [52, 53, 54, 43]。这些方法根据噪声水平将视频分布分解为一系列条件分布, 遵循扩散过程 [55, 56] 或流匹配 [57], 然后使用扩散 Transformer 联合学习所有条件分布。自回归视频扩散 [58, 59, 60, 61, 62, 63] 为这种分解引入了额外的时间维度, 其中神经网络学习在不同噪声水平下对视频下一块的条件概率进行建模, 条件包括之前的视频和当前视频帧的更高噪声版本。

在训练过程中，一些自回归方法采用教师强制，在给定先前干净上下文帧作为条件的情况下，对噪声视频帧进行监督 [58, 59, 60]，但这可能导致令牌利用率低，即只有一小部分令牌得到监督。为了提高令牌效率，提出了其他技术，如渐进式噪声注入 [61] 或使用帧独立噪声（有时采用扩散强制风格）[62, 64, 65]。当将我们的大块设计应用于自回归视频生成时，我们将输入序列格式化为干净块和噪声块交错（见方程 12）。该策略实现了超过 50% 的令牌利用率，并与我们的大块 TTT 实现有效集成，只需更改几行代码即可限制快速权重仅在干净帧块上更新。

7 局限性

我们方法的一个局限性是缺乏旋转不变性。与 Softmax 注意力和线性注意力不同，它们在查询和键的均匀旋转下保持不变（这一特性被相对位置编码如 RoPE 所利用 [42]），我们的 SwiGLU 和线性快速权重组件不具备这一特性。这种缺失的实际影响仍未得到充分探索。

我们在三个任务上进行了实验。尽管这些任务多样且涵盖不同模式，但我们方法的有效性还需要在更多任务上验证。例如，新视角合成任务本质上是一个带有输入姿态信息的三维重建。无姿态重建任务更具挑战性，本文未对此进行探索。

在语言建模任务中，由于计算限制，一些关键方面未被探索。这些方面包括我们的 LaCT 模型的推理能力以及关于参数规模的可扩展性。先前论文表明，基于状态的模型（LaCT 属于此类）的一个主要弱点是其推理能力。然而，推理能力只有在一定的训练计算量下才能获得，因此超出了我们的预算。

最后，对于自回归视频扩散，很难找到一个可靠且可区分的度量来衡量模型的可扩展性。这与使用困惑度（即对数似然损失）的语言建模和使用 PSNR 的新视角合成形成对比。我们在论文中展示了验证损失，这是评估视频生成可扩展性的常见选择。这是视频生成评估的普遍问题，并非我们论文所特有。

8 结论

我们提出了 LaCT，一种新颖的模型架构，它结合了大块测试时训练来捕获长上下文，以及窗口注意力来建模局部结构。我们在三个不同模式的多样化任务上验证了 LaCT——新视角合成、语言建模和自回归视频扩散——并通过与最先进的基线相比取得优越或有竞争力的性能来证明其有效性。LaCT 即使使用原生 PyTorch 实现（仅需几十行代码）也能实现高 GPU 效率，并支持高效扩展状态大小以及在测试时训练模型和优化器中进行更灵活的设计。通过开源代码和权重，我们希望 LaCT 能够推动未来对长上下文建模更高性能架构的研究探索。

致谢

我们感谢 Ziwen Chen 处理 DL3DV 数据集并提供 K-means 聚类。我们感谢 Nathan Carr、Feng Liu、Jianming Zhang 和 Hailin Jin 对本项目的慷慨支持。我们感谢 Haian Jin 和 Zexiang Xu 领导 LVSM 项目。我们感谢 Baqiao Liu 和 Haoran Cai 提供视频数据加载器。我们感谢 Guo Han 提供语言数据集的细节。我们感谢 Jeremy Bernstein 关于 Muon 的讨论。

参考文献

- [1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

- [2] Yu Sun, Xinhao Li, Karan Dalal, Jiarui Xu, Arjun Vikram, Genghan Zhang, Yann Dubois, Xinlei Chen, Xiaolong Wang, Sanmi Koyejo, et al. Learning to (learn at test time): Rnns with expressive hidden states. *arXiv preprint arXiv:2407.04620*, 2024.
- [3] Imanol Schlag, Kazuki Irie, and Jürgen Schmidhuber. Linear transformers are secretly fast weight programmers. In *International Conference on Machine Learning*, pages 9355–9366. PMLR, 2021.
- [4] Ke Alexander Wang, Jiaxin Shi, and Emily B. Fox. Test-time regression: a unifying framework for designing sequence models with associative memory, 2025.
- [5] Ali Behrouz, Peilin Zhong, and Vahab Mirrokni. Titans: Learning to memorize at test time. *arXiv preprint arXiv:2501.00663*, 2024.
- [6] Ali Behrouz, Meisam Razaviyayn, Peilin Zhong, and Vahab Mirrokni. It’s all connected: A journey through test-time memorization, attentional bias, retention, and online optimization, 2025.
- [7] Mahdi Karami and Vahab Mirrokni. Lattice: Learning to efficiently compress the memory. *arXiv preprint arXiv:2504.05646*, 2025.
- [8] Keller Jordan, Yuchen Jin, Vlado Boza, Jiacheng You, Franz Cesista, Laker Newhouse, and Jeremy Bernstein. Muon: An optimizer for hidden layers in neural networks, 2024.
- [9] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3d gaussian splatting for real-time radiance field rendering. *ACM Trans. Graph.*, 42(4):139–1, 2023.
- [10] Songlin Yang, Bailin Wang, Yu Zhang, Yikang Shen, and Yoon Kim. Parallelizing linear transformers with the delta rule over sequence length. *arXiv preprint arXiv:2406.06484*, 2024.
- [11] Yutao Sun, Li Dong, Shaohan Huang, Shuming Ma, Yuqing Xia, Jilong Xue, Jianyong Wang, and Furu Wei. Retentive network: A successor to transformer for large language models. *arXiv preprint arXiv:2307.08621*, 2023.
- [12] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.
- [13] Songlin Yang, Bailin Wang, Yikang Shen, Rameswar Panda, and Yoon Kim. Gated linear attention transformers with hardware-efficient training. In *International Conference on Machine Learning*, pages 56501–56523. PMLR, 2024.
- [14] Zhen Qin, Weigao Sun, Dong Li, Xuyang Shen, Weixuan Sun, and Yiran Zhong. Various lengths, constant speed: Efficient language modeling with lightning attention. In *Forty-first International Conference on Machine Learning*, 2024.
- [15] Songlin Yang, Bailin Wang, Yu Zhang, Yikang Shen, and Yoon Kim. Parallelizing linear transformers with the delta rule over sequence length. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024.
- [16] Tri Dao. Flashattention-2: Faster attention with better parallelism and work partitioning. *arXiv preprint arXiv:2307.08691*, 2023.
- [17] Maxim Milakov and Natalia Gimelshein. Online normalizer calculation for softmax. *arXiv preprint arXiv:1805.02867*, 2018.
- [18] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in neural information processing systems*, 35:16344–16359, 2022.
- [19] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. arxiv e-prints. *arXiv preprint arXiv:1512.03385*, 10:9, 2015.
- [20] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin

- Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurelien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. Llama 2: Open foundation and fine-tuned chat models, 2023.
- [21] Noam Shazeer. Glu variants improve transformer, 2020.
 - [22] Tim Salimans and Durk P Kingma. Weight normalization: A simple reparameterization to accelerate training of deep neural networks. *Advances in neural information processing systems*, 29, 2016.
 - [23] Tri Dao and Albert Gu. Transformers are ssms: Generalized models and efficient algorithms through structured state space duality. *arXiv preprint arXiv:2405.21060*, 2024.
 - [24] Simran Arora, Sabri Eyuboglu, Michael Zhang, Aman Timalsina, Silas Alberti, Dylan Zinsley, James Zou, Atri Rudra, and Christopher Ré. Simple linear attention language models balance the recall-throughput tradeoff. *arXiv preprint arXiv:2402.18668*, 2024.
 - [25] Weizhe Hua, Zihang Dai, Hanxiao Liu, and Quoc Le. Transformer quality in linear time. In *International conference on machine learning*, pages 9099–9117. PMLR, 2022.
 - [26] Tsendsuren Munkhdalai, Manaal Faruqui, and Siddharth Gopal. Leave no context behind: Efficient infinite context transformers with infini-attention, 2024.
 - [27] Leonard McMillan and Gary Bishop. Plenoptic modeling: An image-based rendering system. In *Seminal Graphics Papers: Pushing the Boundaries, Volume 2*, pages 433–440. 2023.
 - [28] Marc Levoy and Pat Hanrahan. Light field rendering. In *Seminal Graphics Papers: Pushing the Boundaries, Volume 2*, pages 441–452. 2023.
 - [29] Haian Jin, Hanwen Jiang, Hao Tan, Kai Zhang, Sai Bi, Tianyuan Zhang, Fujun Luan, Noah Snavely, and Zexiang Xu. Lvsrn: A large view synthesis model with minimal 3d inductive bias. *arXiv preprint arXiv:2410.17242*, 2024.
 - [30] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*, 2020.
 - [31] Matt Deitke, Dustin Schwenk, Jordi Salvador, Luca Weihs, Oscar Michel, Eli VanderBilt, Ludwig Schmidt, Kiana Ehsani, Aniruddha Kembhavi, and Ali Farhadi. Objaverse: A universe of annotated 3d objects. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 13142–13153, 2023.
 - [32] Kai Zhang, Sai Bi, Hao Tan, Yuanbo Xiangli, Nanxuan Zhao, Kalyan Sunkavalli, and Zexiang Xu. Gs-lrm: Large reconstruction model for 3d gaussian splatting. In *European Conference on Computer Vision*, pages 1–19. Springer, 2024.
 - [33] Laura Downs, Anthony Francis, Nate Koenig, Brandon Kinman, Ryan Hickman, Krista Reymann, Thomas B McHugh, and Vincent Vanhoucke. Google scanned objects: A high-quality dataset of 3d scanned household items. In *2022 International Conference on Robotics and Automation (ICRA)*, pages 2553–2560. IEEE, 2022.
 - [34] Lu Ling, Yichen Sheng, Zhi Tu, Wentian Zhao, Cheng Xin, Kun Wan, Lantao Yu, Qianyu Guo, Zixun Yu, Yawen Lu, et al. D3dv-10k: A large-scale scene dataset for deep learning-based 3d vision. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 22160–22169, 2024.
 - [35] Andrew Jaegle, Felix Gimeno, Andy Brock, Oriol Vinyals, Andrew Zisserman, and Joao Carreira. Perceiver: General perception with iterative attention. In *International conference on machine learning*, pages 4651–4664. PMLR, 2021.
 - [36] Chen Ziwen, Hao Tan, Kai Zhang, Sai Bi, Fujun Luan, Yicong Hong, Li Fuxin, and Zexiang Xu. Long-lrm: Long-sequence large reconstruction model for wide-coverage gaussian splats. *arXiv preprint arXiv:2410.12781*, 2024.
 - [37] Together AI. Long data collections database, 2024.
 - [38] Zhixuan Lin, Evgenii Nikishin, Xu He, and Aaron Courville. Forgetting transformer: Softmax attention with a forget gate. In *The Thirteenth International Conference on Learning Representations*, 2025.
 - [39] Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekeshe, Fei Jia, and Boris Ginsburg. Ruler: What’s the real context size of your long-context language models? In *First Conference on Language Modeling*, 2024.

- [40] Wenhan Xiong, Jingyu Liu, Igor Molybog, Hejia Zhang, Prajwal Bhargava, Rui Hou, Louis Martin, Rashi Rungta, Karthik Abinav Sankararaman, Barlas Oguz, Madian Khabsa, Han Fang, Yashar Mehdad, Sharan Narang, Kshitiz Malik, Angela Fan, Shruti Bhosale, Sergey Edunov, Mike Lewis, Sinong Wang, and Hao Ma. Effective long-context scaling of foundation models, 2023.
- [41] Xin Men, Mingyu Xu, Bingning Wang, Qingyu Zhang, Hongyu Lin, Xianpei Han, and Weipeng Chen. Base of rope bounds context length. *arXiv preprint arXiv:2405.14591*, 2024.
- [42] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding, 2023.
- [43] Ang Wang, Baole Ai, Bin Wen, Chaojie Mao, Chen-Wei Xie, Di Chen, Feiwu Yu, Haiming Zhao, Jianxiao Yang, Jianyuan Zeng, et al. Wan: Open and advanced large-scale video generative models. *arXiv preprint arXiv:2503.20314*, 2025.
- [44] Patrick Esser, Sumith Kulal, Andreas Blattmann, Rahim Entezari, Jonas Müller, Harry Saini, Yam Levi, Dominik Lorenz, Axel Sauer, Frederic Boesel, et al. Scaling rectified flow transformers for high-resolution image synthesis, 2024. URL <https://arxiv.org/abs/2403.03206>, 2, 2024.
- [45] Ali Behrouz, Meisam Razaviyayn, Peilin Zhong, and Vahab Mirrokni. It’s all connected: A journey through test-time memorization, attentional bias, retention, and online optimization. *arXiv preprint arXiv:2504.13173*, 2025.
- [46] Weizhe Hua, Zihang Dai, Hanxiao Liu, and Quoc V. Le. Transformer quality in linear time, 2022.
- [47] Xuezhe Ma, Chunting Zhou, Xiang Kong, Junxian He, Liangke Gui, Graham Neubig, Jonathan May, and Luke Zettlemoyer. Mega: Moving average equipped gated attention. In *The Eleventh International Conference on Learning Representations*, 2023.
- [48] Xuezhe Ma, Xiaomeng Yang, Wenhan Xiong, Beidi Chen, LILI YU, Hao Zhang, Jonathan May, Luke Zettlemoyer, Omer Levy, and Chunting Zhou. Megalodon: Efficient LLM pretraining and inference with unlimited context length. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024.
- [49] DeLesley Hutchins, Imanol Schlag, Yuhuai Wu, Ethan Dyer, and Behnam Neyshabur. Block-recurrent transformers. In Alice H. Oh, Alekh Agarwal, Danielle Belgrave, and Kyunghyun Cho, editors, *Advances in Neural Information Processing Systems*, 2022.
- [50] Ben Mildenhall, Pratul P Srinivasan, Matthew Tancik, Jonathan T Barron, Ravi Ramamoorthi, and Ren Ng. Nerf: Representing scenes as neural radiance fields for view synthesis. *Communications of the ACM*, 65(1):99–106, 2021.
- [51] Ruoshi Liu, Rundi Wu, Basile Van Hoorick, Pavel Tokmakov, Sergey Zakharov, and Carl Vondrick. Zero-1-to-3: Zero-shot one image to 3d object. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 9298–9309, 2023.
- [52] Tim Brooks, Bill Peebles, Connor Holmes, Will DePue, Yufei Guo, Li Jing, David Schnurr, Joe Taylor, Troy Luhman, Eric Luhman, Clarence Ng, Ricky Wang, and Aditya Ramesh. Video generation models as world simulators. 2024.
- [53] Zhuoyi Yang, Jiayan Teng, Wendi Zheng, Ming Ding, Shiyu Huang, Jiazheng Xu, Yuanming Yang, Wenyi Hong, Xiaohan Zhang, Guanyu Feng, et al. Cogvideox: Text-to-video diffusion models with an expert transformer. *arXiv preprint arXiv:2408.06072*, 2024.
- [54] A Polyak, A Zohar, A Brown, A Tjandra, A Sinha, A Lee, A Vyas, B Shi, CY Ma, CY Chuang, et al. Movie gen: A cast of media foundation models, 2025. URL <https://arxiv.org/abs/2410.13720>, page 51, 2024.
- [55] Jascha Sohl-Dickstein, Eric Weiss, Niru Maheswaranathan, and Surya Ganguli. Deep unsupervised learning using nonequilibrium thermodynamics. In *International conference on machine learning*, pages 2256–2265. pmlr, 2015.
- [56] Yang Song, Jascha Sohl-Dickstein, Diederik P Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-based generative modeling through stochastic differential equations. *arXiv preprint arXiv:2011.13456*, 2020.
- [57] Yaron Lipman, Ricky TQ Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. *arXiv preprint arXiv:2210.02747*, 2022.

- [58] Eloi Alonso, Adam Jelley, Vincent Micheli, Anssi Kanervisto, Amos J Storkey, Tim Pearce, and François Fleuret. Diffusion for world modeling: Visual details matter in atari. *Advances in Neural Information Processing Systems*, 37:58757–58791, 2024.
- [59] Yang Jin, Zhicheng Sun, Ningyuan Li, Kun Xu, Hao Jiang, Nan Zhuang, Quzhe Huang, Yang Song, Yadong Mu, and Zhouchen Lin. Pyramidal flow matching for efficient video generative modeling. *arXiv preprint arXiv:2410.05954*, 2024.
- [60] Dani Valevski, Yaniv Leviathan, Moab Arar, and Shlomi Fruchter. Diffusion models are real-time game engines. *arXiv preprint arXiv:2408.14837*, 2024.
- [61] David Ruhe, Jonathan Heek, Tim Salimans, and Emiel Hooeboom. Rolling diffusion models. In *International Conference on Machine Learning*, pages 42818–42835. PMLR, 2024.
- [62] Tianwei Yin, Qiang Zhang, Richard Zhang, William T Freeman, Fredo Durand, Eli Shechtman, and Xun Huang. From slow bidirectional to fast causal video generators. *arXiv preprint arXiv:2412.07772*, 2024.
- [63] Kiwhan Song, Boyuan Chen, Max Simchowitz, Yilun Du, Russ Tedrake, and Vincent Sitzmann. History-guided video diffusion. *arXiv preprint arXiv:2502.06764*, 2025.
- [64] Boyuan Chen, Diego Martí Monsó, Yilun Du, Max Simchowitz, Russ Tedrake, and Vincent Sitzmann. Diffusion forcing: Next-token prediction meets full-sequence diffusion. *Advances in Neural Information Processing Systems*, 37:24081–24125, 2024.
- [65] Sand-AI. Magi-1: Autoregressive video generation at scale, 2025.
- [66] Richard Zhang, Phillip Isola, Alexei A Efros, Eli Shechtman, and Oliver Wang. The unreasonable effectiveness of deep features as a perceptual metric. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 586–595, 2018.
- [67] Alex Henry, Prudhvi Raj Dachapally, Shubham Shantaram Pawar, and Yuxuan Chen. Query-key normalization for transformers. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 4246–4253, 2020.
- [68] Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, Shawn Presser, and Connor Leahy. The pile: An 800gb dataset of diverse text for language modeling. *arXiv preprint arXiv:2101.00027*, 2020.
- [69] Zhe Chen, Jiannan Wu, Wenhai Wang, Weijie Su, Guo Chen, Sen Xing, Muyan Zhong, Qinglong Zhang, Xizhou Zhu, Lewei Lu, et al. Internvl: Scaling up vision foundation models and aligning for generic visual-linguistic tasks. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 24185–24198, 2024.
- [70] Junxiong Wang, Daniele Paliotta, Avner May, Alexander Rush, and Tri Dao. The mamba in the llama: Distilling and accelerating hybrid models. *Advances in Neural Information Processing Systems*, 37:62432–62457, 2024.
- [71] Horace He, Driss Guessous, Yanbo Liang, and Joy Dong. Flexattention: The flexibility of pytorch with the performance of flashattention. *PyTorch Blog*, 8, 2024.
- [72] Wenliang Zhao, Lujia Bai, Yongming Rao, Jie Zhou, and Jiwen Lu. Unipc: A unified predictor-corrector framework for fast sampling of diffusion models. *Advances in Neural Information Processing Systems*, 36:49842–49869, 2023.
- [73] Sam Ade Jacobs, Masahiro Tanaka, Chengming Zhang, Minjia Zhang, Shuaiwen Leon Song, Samyam Rajbhandari, and Yuxiong He. Deepspeed ulysses: System optimizations for enabling training of extreme long sequence transformer models. *arXiv preprint arXiv:2309.14509*, 2023.
- [74] Jiarui Fang and Shangchun Zhao. Usp: A unified sequence parallelism approach for long context generative ai. *arXiv preprint arXiv:2405.07719*, 2024.

A LaCT 模型实现细节

状态尺寸计算。受近期大语言模型（LLM）进展的启发，本文采用不带偏差项（bias）的 SwiGLU-MLP [21] 作为快权重网络。本文的快权重由三个权重矩阵 $W = \{W_1, W_2, W_3\}$ 组成，快权重模型的前向传播过程为：

$$f_W(x) = W_2 [\text{SiLU}(W_1 x) \circ (W_3 x)] \quad (13)$$

其中 $\circ$ 表示逐元素乘法。本文定义 hd 为头维度， nh 为头数，SwiGLU-MLP 的中间维度为 $hd \times r$ ，其中 r 为缩放乘数。当 $r > 1$ 时，中间维度超过输入头维度，这是当前大语言模型中的常见做法。因此，矩阵 $W_1, W_2 \in \mathbb{R}^{hd \times hd}$ 和 $W_3 \in \mathbb{R}^{hd \times hd \times r}$ 随之确定。最终，总状态尺寸变为 $nh \times hd \times hd * r$ 。鉴于所有头的总头维度通常等于模型维度 d (i.e., $nh \times hd = d$ ，总状态尺寸简化为：状态尺寸 $= \frac{d^2}{nh} * r$ 。

$$nh * r. \quad (14)$$

因此，本文可以通过减少头数或增大中间维度乘数来增加状态尺寸。

浮点运算次数（FLOPs）计算。当使用负点积损失作为在线测试时训练目标时，无需计算 $f_W(v)$ 的最终结果。仅需在使用键 k 运行前向传播时计算 $W_1 v, W_3 v$ ，因此前向传播中包含两次矩阵乘法。计算梯度时，包含四次矩阵乘法。在最终使用查询的前向传播中，包含三次矩阵乘法。因此，对于 n 个标记，总浮点运算次数为：

$$\text{FLOPs} = 4n \frac{d^2}{nh} r + 8n \frac{d^2}{nh} r + 6n \frac{d^2}{nh} r = 18n \frac{d^2}{nh} r = 6 * \text{State Size} \quad (15)$$

模型初始化。本文以标准差 0.02 随机初始化线性层。对于可学习的初始快权重，本文以标准差 $\frac{1.0}{\sqrt{nh}}$ 扇入（fan-in）进行初始化。具体而言，在 SwiGLU FFN 快权重中，矩阵 w_1 和 w_3 的扇入设为头维度，而 w_2 的扇入为 SwiGLU FFN 快权重的中间维度。此外，当在 LaCT 层内引入局部窗口注意力时，本文额外引入四个可学习嵌入：两个用于查询和值的缩放参数及两个再偏移参数。本文将缩放嵌入初始化为 1，将再偏移嵌入初始化为 0。

Muon 的细节。Muon [8] 是最近提出的一种优化器，在更新矩阵权重时对矩阵梯度进行正交化。它利用牛顿-舒尔茨迭代来实现正交化。给定矩阵梯度 G ，Muon 首先将其归一化为 $G_0 = G/|G|_F$ ，然后迭代地应用：

$$\mathbf{G}_k = a\mathbf{G}_{k-1} + b(\mathbf{G}_{k-1}\mathbf{G}_{k-1}^T)\mathbf{G}_{k-1} + c(\mathbf{G}_{k-1}\mathbf{G}_{k-1}^T)^2\mathbf{G}_{k-1}, \quad (16)$$

其中常数 a, b, c 经过精心选择以获得最佳收敛性。遵循原始实现，我们设置 $a = 3.4445, b = -4.7750, c = 2.0315$ ，并执行五次迭代。

每次 Muon 迭代需要三次矩阵乘法，导致每个快速权重头的计算成本为 $2hd^3r + 2hd^3 + 2hd^3r = hd^3(4r + 2)$ FLOPs。因此，所有快速权重上五次迭代的总计算量为：

$$5 \times nh \times hd^3 \times (4r + 2). \quad (17)$$

对于 $r = 1$ （头维度和中间维度相等）的情况，总计算成本简化为：状态大小。这表明，只有当在线块大小超过 $\frac{5}{3}hd$ 时， $30 \times nh \times hd^3 = 30 \times hd \times$ Muon 的计算开销才会变得不如计算令牌输出显著。

旋转不变性。Softmax 注意力和线性注意力表现出旋转不变性：用相同的旋转矩阵旋转查询和键不会改变输出。这一性质也被用于开发相对位置编码，如 RoPE [42]。相比之下，我们的 SwiGLU 和线性快速权重组件不具备这一性质。

Algorithm 1 Large Chunk Test-Time Training Layer Pseudocode

```
def apply_fw(fast_weight, q):
    w1, w2, w3 = fast_weight
    hidden = silu(matmul(q, w1)) * matmul(q, w3) # [b, 1, dh] = [b, 1, d] x [b, d, dh]
    return matmul(hidden, w2)

def update(fast_weight, k, v, lr, use_muon=True):
    """
    Fast-weight update for a SwiGLU MLP using chunk of tensors.
    Args:
        fast_weight : tuple(w1, w2, w3) with shapes: w1, w3: [b, d, dh]; w2: [b, dh, d]
        k, v : key / value tensor of shape [b, 1, d]
        lr : per-token learning rates of shape [b, 1, 3] -> (lr1, lr2, lr3)
        use_muon : whether to apply Muon to orthogonalize the update
    Note:
        The head dimension for input tensors k, v, lr are assumed to be merged into the batch dimension.
        This is not necessary, but simplifies shape annotation in this pseudocode.
    """
    # Forward with k:
    gate_before_act = matmul(k, w1) # [b, 1, dh] = [b, 1, d] x [b, d, dh]
    hidden_before_gate = matmul(k, w3) # [b, 1, dh] = [b, 1, d] x [b, d, dh]
    hidden = silu(gate_before_act) * hidden_before_gate

    # Backward:
    dhidden = matmul(v, w2.transpose(-1, -2)) # [b, 1, dh] = [b, 1, d] x [b, d, dh]
    dhidden_before_gate = dhidden * silu(gate_before_act)
    dgate = dhidden * hidden_before_gate
    dgate_before_act = silu_backprop(dgate, gate_before_act)

    # Compute gradients:
    w2.grad = -matmul(hidden.transpose(-1, -2), v * lr2) # [b, dh, d] = [b, dh, 1] x [b, 1, d]
    # [b, d, dh] = [b, d, 1] x [b, 1, dh]
    w1.grad = -matmul((k * lr1).transpose(-1, -2), dgate_before_act)
    w3.grad = -matmul((k * lr3).transpose(-1, -2), dhidden_before_gate)

    # Weight update
    if use_muon:
        for w in fast_weight:
            w.grad = zeropower_via_newtonschulz5(w.grad)
    for w in fast_weight:
        w = (w - w.grad) / (w - w.grad).norm(dim=1) * w.norm(dim=1)
    return fast_weight

def silu_backprop(dy, x):
    sigma = sigmoid(x)
    return dy * sigma * (1 + x * (1 - sigma))

##### MultiHead LaCT layer #####
# x: input sequence [b, 1, d], b is the batch dim, 1 is length, d is model dimension
# fast_weight: tuple of initial fast weights-(w1, w2, w3); w1, w3 of shape [nh, d, dh], w2: [nh, dh, d]
# ttt_config: list of (operation, begin, end) tuples
qkv = silu(LinearQKV(x)) # [b, 1, d * 3]
qkv = rearrange(qkv, 'b 1 (nh hd)' -> (b nh) 1 hd', nh=num_heads).split(3) # merge heads into batch dim
q, k = q / q.norm(-1), k / k.norm(-1)
lr = softplus(LinearLR(x) + const_lr_bias) # [b, 1, 3 * num_heads]
lr = rearrange(lr, 'b 1 (nh 3)' -> (b nh) 1 3', nh=num_heads)
fast_weight = repeat(fast_weight, dim=0, repeat=b) # [nh, ...] -> [b * nh, ...]

o = zeros_like(v) # [b * nh, 1, hd]
for mode, begin, end in ttt_config:
    qi, ki, vi, lri = q[:, begin:end], k[:, begin:end], v[:, begin:end], lr[:, begin:end]

    if mode == "update_then_apply": # figure-3(a, b) bidirectional attention
        fast_weight = update(fast_weight, ki, vi, lri, use_muon)
        o[:, begin:end] = apply_fw(fast_weight, qi)

    elif mode == "apply_then_update": # figure-3(b) shifted block-wise causal
        o[:, begin:end] = apply_fw(fast_weight, qi)
        fast_weight = update(fast_weight, ki, vi, lri, use_muon)

    elif mode == "update_only":
        fast_weight = update(fast_weight, ki, vi, lri, use_muon)

    elif mode == "apply_only":
        o[:, begin:end] = apply_fw(fast_weight, qi)

o = RMSNorm(o) # per-head norm
o = LinearOutput(rearrange(o, '(b nh) 1 hd -> b 1 (nh hd)', nh=num_heads))
return o
```

Algorithm 2 LaCT Layer with In-Layer Hybrid Window Attention Pseudocode

```
# Input:
# x: input sequence [b, l, d], b is the batch dim, l is length, d is model dimension
# fast_weight: tuple of initial fast weights-(w1, w2, w3); w1, w3 of shape [d, dh], w2 of shape [dh, d]
# ttt_config: list of (operation, begin, end) tuples

q, k, v = LinearQKV(x).split(3)

#### Local quadratic-cost window attention
attn_q = q * learnable_q_scale + learnable_q_offset # per-channel rescale and shift
attn_k = k * learnable_k_scale + learnable_k_offset # per-channel rescale and shift
attn_o = local_softmax_multihead_attn(attn_q, attn_k, v, attn_mask)

#### large chunk test-time training for long memory
q, k = rearrange(q, k, `b l (nh hd) -> (b nh) l hd`, nh=num_heads)
q, k = silu(q), silu(k)
q, k = q / q.norm(-1), k / k.norm(-1)
lr = softplus(LinearLR(x)) # [b, l, 3 * num_heads]
lr = rearrange(lr, `b l (nh 3) -> (b nh) l 3`, nh=num_heads)

# Perform update and apply_fw operations iteratively over chunks of tokens.
lact_o = lact(fast_weight, q, k, v, lr, ttt_config)
lact_o = RMSNorm(lact_o)

scale_per_head = rearrange(silu(Linear(x)), `b l nh -> (b nh) l 1`, nh=num_heads)
lact_o = lact_o * scale_per_head
lact_o = rearrange(lact_o, `(b nh) l hd -> b l (nh hd)`, nh=num_heads)

#### Merge attention results (shape: [b, l, d])
o = attn_o + lact_o

o = LinearOutput(o)

return o
```

为测试时优化器实现动量。Muon 默认使用动量。遵循 Titans [5]，我们在测试时优化器中通过从每个标记预测一个标量动量 β_i 来实现动量：

$$\beta_i = \sigma(\text{Linear}(x_i)), \quad (19)$$

其中 σ 是 Sigmoid 函数。该 β_i 随后在块中的所有标记上取平均，平均动量按如下方式应用：

$$\begin{aligned} g &\leftarrow \sum_i^b \eta_i \nabla_W \mathcal{L}(f_W(k_i), v_i), \\ M &\leftarrow M(\sum_i^b \beta_i / b) + g, \\ W &\leftarrow \text{weight-update}(W, M), \end{aligned} \quad (20)$$

其中权重更新可以是简单减法后接 L2 归一化（如方程 8 所示），或减法前进行 Muon 更新（如方程 9 所示）。

A.1 伪代码

完整 LaCT 层的伪代码见算法 1。关于如何在每层内部混合局部窗口注意力与共享查询、值、嵌入的细节，见算法 2 的伪代码。

B 面向 N 维数据的 LaCT 架构细节

B.1 用于新视角合成的 LaCT 架构

新视角合成（Novel View Synthesis, NVS）从新视角渲染静态场景的图像。形式上，给定一组 N 输入位姿图像 $\{(I_i, P_i)\}_{i=1}^N$ 的静态场景，其中 $I_i \in \mathbb{R}^{H \times W \times 3}$ 是 RGB 图像， P_i 是其对应的相机位姿，模型需要从通常与输入视图不重叠的新相机位姿 P_{novel} 合成新图像。

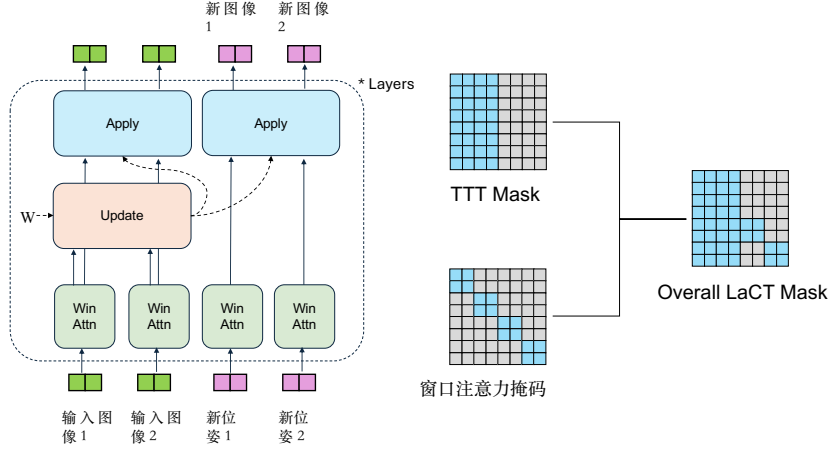


图 9: 用于新视角合成的详细 LaCT 模型。虚线表示快速权重的流动。实线表示令牌的流动。窗口注意力在单张图像内是双向的，无论是输入图像还是新目标图像。TTT 在所有输入令牌上更新，并应用于所有令牌。

传统 3D 视觉方法通常通过重建和渲染来解决新视角合成任务。重建将带姿态的输入压缩为紧凑表示，然后渲染器从该表示中渲染出新视角。本文方法模拟了这种流程，首先通过 LaCT 中的“更新操作”（第 3.1 节）将带姿态的输入图像压缩为快速权重，然后利用“应用操作”从压缩权重中的信息渲染出新视角图像。

具体而言，本文首先将输入和输出转换为令牌。每个视角的相机姿态 F 以每个像素的密集光线信息表示（通常来自相机内参和外参），i.e., $P = (\text{光线}_o, \text{光线}_d)$ 。光线 $o \in \mathbb{R}^{H \times W \times 3}$ 是光线原点的三维坐标，光线 $d \in \mathbb{R}^{H \times W \times 3}$ 是光线的方向。本文遵循 GS-LRM[32] 使用普吕克光线嵌入。普吕克光线嵌入计算光线原点与光线方向的叉积以进行归一化。最终的位置嵌入是光线原点、光线方向以及上述两者的叉积的拼接： $[\text{光线}_o, \text{光线}_d, \text{光线}_o \times \text{光线}_d]$ 。本文将光线原点加入嵌入中，因为同一光线上不同的原点可能由于遮挡而导致不同的颜色。随后，本文使用分块和两个不同的线性层将 RGB 图和光线图（即位置嵌入）转换为令牌。对于带姿态的 RGB 输入图像，本文简单地将 RGB 嵌入和姿态嵌入相加作为模型输入。对于新视角相机，本文仅使用姿态嵌入。

我们在图 9 中展示了 NVS 的 LaCT 模块设计。我们首先对每张图像（无论是输入图像还是新视角目标图像）应用注意力机制。对于属于同一图像的标记，注意力是双向的，并且在不同图像之间相互独立。然后，TTT 更新操作应用于所有输入标记，即属于所有输入图像的所有标记。更新后的权重随后应用于所有标记。图 9 中的两个更新模块采用相同的更新后快速权重，因此可以合并，为了清晰起见，我们保留了“应用”模块。需要注意的是，原始的 NVS 任务定义是独立渲染新视角的。我们在这里将多个新视角姿态及其对应图像放在单个数据点中进行监督，以提高训练效率。根据该设计，新视角图像彼此独立，如图 9 右侧的“整体 LaCT 掩码”所示。为了清晰起见，省略了层归一化层、残差连接和前馈网络。该模块重复若干层以构成完整模型。整体模型在很大程度上遵循 LVSM 中编码器-解码器模型的设计 [29]，不同之处在于我们使用 TTT 替代变换器进行长上下文建模。

在推理过程中实际使用该模型进行 NVS 任务时，我们首先利用所有输入图像获得更新后的快速权重。然后，在渲染过程中（即将新相机姿态转换为新图像的过程）我们不会改变快速权重。渲染过程中的 LaCT 将类似于 ViT（视觉变换器）架构，尽管它有两个前馈网络：来自快速权重的前馈网络存储场景信息，而来自慢权重的前馈网络

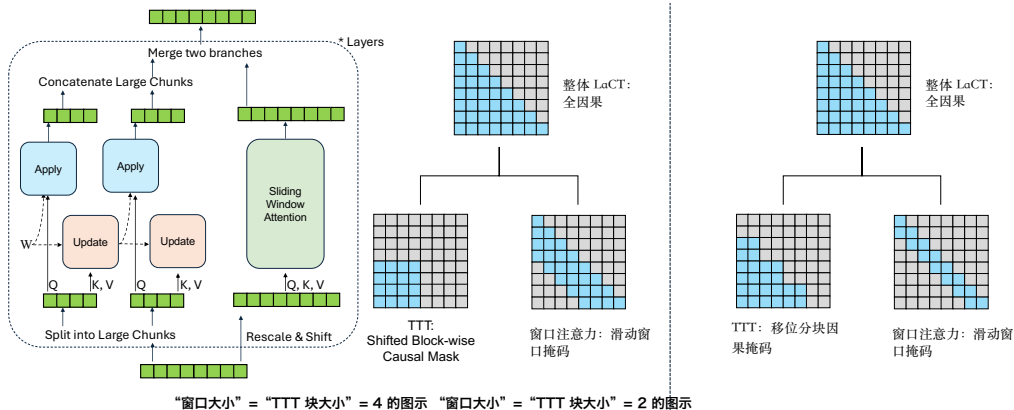


图 10: 用于语言模型的详细 LaCT 模型。虚线表示快速权重的流动。实线表示标记的流动。我们以 TTT 块大小 4 或 2 进行说明，而 LaCT 中的实际块大小超过 2048。我们对窗口注意力和 TTT 模块采用并行设计，并共享 QKV。整体掩码为因果掩码。

(即图 2 中的 FFN) 存储了世界知识，如物理渲染规则。

B.2 语言模型的拉普拉斯上下文变换架构

自回归语言模型 (Autoregressive Language Models, LM) 根据其历史上下文预测下一个词元的分布 $p_\theta(x_n|x_1, \dots, x_{n-1})$ 。这是通过链式法则

$p_\theta(x_1, \dots, x_n) = p_\theta(x_1)p_\theta(x_2|x_1) \dots p_\theta(x_n|x_1, \dots, x_{n-1})$ 对完整序列分布 $p_\theta(x_1, \dots, x_n)$ 的因式分解。因此，它需要词元级别的因果掩码 (如图 10 右上角所示)，这也是拉普拉斯上下文变换 (LaCT) 中大块设计的主要难点。我们结合使用带有“移位因果块掩码” (此前在图 3c 中介绍) 的测试时训练 (TTT) 层和滑动窗口注意力来促进这一点。通过移动 TTT 的掩码，排除了来自未来词元的信息泄漏。如图 10 右侧所示，整体依赖掩码是 TTT 掩码和滑动窗口注意力掩码的并集。为了实现无气泡的词元级因果掩码，唯一的要求是滑动窗口注意力的窗口大小大于或等于 TTT 的块大小。我们展示了两个此类掩码的示例，其中“窗口大小”=“TTT 块大小”= 2 或 4。在我们的实现中，实际块大小和注意力窗口大小超过 2048，以获得更好的利用率和状态大小缩放 (在第 2.2 节中讨论)。

如图 10 最左侧所示，我们采用 TTT 层和滑动窗口注意力的并行设计，以节省模型参数数量和计算浮点运算次数 (FLOPs)。具体而言，查询 (Q)、键 (K) 和值 (V) 在 TTT 层和窗口注意力之间共享。滑动窗口注意力是一种在目标词元开始的历史上具有恒定窗口大小的注意力。对于 TTT 层，我们首先使用初始化的快速权重 (即尚未更新) 对第一个块执行应用操作。“应用”操作之后是对第一个块的“更新”操作。这样，“应用”操作就不会看到当前块内的信息，从而避免泄漏块内未来词元的信息。或者，在后续块上使用“应用”后跟“更新”完成图 10 所示的所需“移位块级因果掩码”。有关并行设计的详细信息，请参阅伪代码算法 2。

我们对 TTT 层和窗口注意力均采用多头设计，尽管两者的头数不同。根据经验，我们为 TTT 层采用较少的头数 (即更大的头维度)，以支持更大的状态规模，因为在我们设计中状态规模与头维度成正比 (附录 A 及方程 14)。默认情况下，我们在语言模型实验中使用四个头。对于位置编码，我们使用与窗口注意力分支相同的旋转位置编码。

B.3 用于自回归视频扩散的 LaCT 架构

分块自回归视频扩散通过迭代去噪连续的视频帧块来生成视频，并以先前生成的干净帧为条件。每个块可以包含若干视频帧，并张成数千个视觉标记。我们通过交错使用教师强制训练

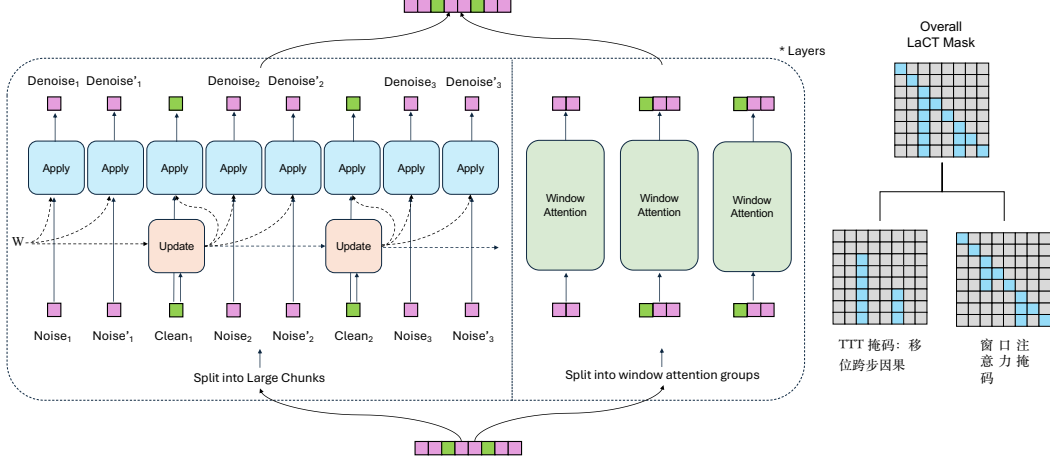


图 11: 用于扩散模型自回归视频生成的详细 LaCT 模型。紫色标记为用于训练扩散模型的噪声帧块。绿色标记为干净帧块。每个标记在 TTT 中是一个大块, 例如 3 个连续帧共 4680 个标记。噪声与噪声'是同一干净帧上具有独立高斯噪声和时间戳的两个噪声帧。我们同时去噪它们以提高利用率。虚线表示快速权重的流动。实线表示标记的流动。我们对窗口注意力和 TTT 块采用并行设计, 共享查询键值。TTT 掩码是移位的 (即以应用开始) 跨步因果。窗口注意力排除了从干净帧到未来噪声帧的注意力, 也排除了独立噪声帧之间的注意力。

噪声和干净帧块。具体来说, 一个包含 N 个帧块的视频被结构化:

$$S = [X_1^{\text{noise}}, X_1, X_2^{\text{noise}}, X_2, \dots, X_N^{\text{noise}}] \quad (21)$$

其中每个噪声块 X_i^{noise} 是通过将单位高斯噪声 ϵ 加到第 i 个干净视频块上产生的, 即 $X_i^{\text{noise}} = X_i(1 - t_i) + \epsilon t_i$, 而 $t_i \in [0, 1]$ 表示块独立噪声的强度。然而, 与之前采用渐进式 [61] 或帧独立噪声策略 [62] (如扩散强制 [64]) 的方法相比, 我们在方程 21 中的教师强制公式仅使用整个序列约 50% 的标记来计算去噪损失。为了提高标记利用率, 我们考虑一种替代方法, 在训练序列中为每个视频块重复两个噪声级别, 如下所示:

$$S = [X_1^{\text{noise}}, X_1^{\text{noise}*}, X_1, X_2^{\text{noise}}, X_2^{\text{noise}*}, X_2, \dots, X_N^{\text{noise}}, X_N^{\text{noise}*}], \quad (22)$$

其中 X_i^{noise} 和 $X_i^{\text{noise}*}$ 表示应用于每个干净视频块 X_i 的两种不同噪声水平。这将令牌利用率从 50% 提高到约 67%。虽然更多重复可以进一步提高令牌利用率, 但也会降低训练样本多样性; 因此, 我们将重复限制为两次。在训练 13 亿参数视频扩散模型处理五秒视频时, 我们采用这种重复策略。

为了处理这些交错的噪声和干净块, LaCT 快速权重仅使用干净视频块进行更新。这些更新后的权重随后应用于当前干净块和后续噪声块。集成的局部窗口注意力使用两个帧块的窗口大小, 并采用块级因果掩码。该掩码允许噪声块仅关注自身及紧邻的前一个干净块。通过这种方式, 我们在每个块内保持双向依赖, 并在块间保持因果依赖。

与语言模型实验类似, 我们将 TTT 和局部窗口注意力集成到同一层中。图 11 展示了用于自回归视频生成的这一设计。图中所示的输入序列遵循方程 22, 每个视频块以不同噪声水平重复两次。图中的粉色表示噪声块, 绿色表示干净块。

C 实验细节

C.1 新视角合成

数据集与度量。 我们在物体级和场景级数据集上评估我们的方法。我们使用 Objaverse 数据集 [31] 进行物体级训练，并按照 LVSM[29] 和 GS-LRM[32] 的设置，为每个物体渲染 32 个随机视角。训练后，我们在 Google Scanned Objects (GSO) 数据集 [33] 上进行评估，分辨率为 256×256 和 512×512 。为确保评估结果稳定，我们使用固定视角而非训练中的随机视角进行渲染。每次评估涉及每个物体 4–48 个输入视角和 8 个新视角。对于场景级评估，我们采用具有挑战性的 DL3DV 场景数据集 [34]，该数据集包含超过 11K 个训练场景和 140 个测试场景，每个场景约有 300 个视 960×536^3 角。评估在分辨率为下进行。我们在论文的主要图中报告峰值信噪比 (PSNR)，其他度量结构相似性指数 (SSIM) 和 LPIPS[66] 可在下表中找到。

对于 DL3DV 评估，我们遵循原始论文 [34] 和 LongLRM[36] 的做法，从完整视频序列中每 8 帧选取一帧作为目标帧。输入帧来自所有帧的 K 均值聚类，如 [36] 所述。

模型细节。 我们的模型由 24 个堆叠的 LaCT 块组成，每个块的模型维度为 768。该块的细节在 C.1 节中说明：除非另有说明，我们使用单头快速权重 SwiGLU-MLP，隐藏维度为 1536。窗口注意力有 12 个头，头维度为 64，并配备 QK 归一化 [67]。前馈网络的中间隐藏维度为 3072。模型总参数量为 312M，其中 84M 为快速权重（即每个块 $6d^2$ ）。我们通过将算法 1 中的‘const_lr_bias’设置为 $\text{softplus}(\text{const_lr_bias}) = 0.01$ 来初始化快速权重学习率 0.01。由于我们在 NVS 的快速权重更新中使用了 Muon，LaCT 对学习率尺度不敏感，如 3.2 节所述。

基线。 对于物体级评估，我们与两个基线进行比较，包括一个全注意力模型和一个 Perceiver 风格的寄存器注意力模型 [35]。在全注意力基线中，我们将模型中的 TTT 层替换为块级因果注意力层，其中输入令牌双向交互，新视角令牌对输入令牌进行交叉注意力。这种设计类似于我们方法在第 4.1 节中描述的预填充和并行解码策略，输入令牌的键值缓存作为新视角渲染的场景表示。在 Perceiver 风格模型中，我们将一半的 TTT 层替换为输入到寄存器的全注意力层，另一半替换为寄存器到新视角的交叉注意力层。该模型首先将输入令牌压缩为一组常量寄存器令牌，然后通过关注寄存器来解码新视角令牌。对于场景级评估，我们与最先进的长序列三维重建工作 LongLRM[36] 进行比较，该工作将 Mamba[12] 与全注意力混合以预测三维高斯泼溅 [9]。我们还包括与纯基于优化的三维高斯泼溅方法的比较。表 2 比较了基线模型和我们的模型的计算复杂度。

训练细节。 在物体级实验中，我们首先以 256×256 分辨率、8 个输入视图和 8 个新视图的设置，使用 671B 个标记训练所有模型。随后，我们以 512×512 分辨率对它们进行微调，额外使用 587B 个标记。对于场景数据集，我们首先以 128×128 分辨率、32 个输入视图和 32 个新视图对模型进行预训练，使用 1.5 T 个标记，然后逐步在更大分辨率、更大视场角和更多输入视图下进行微调。微调始终采用非正方形视场角以匹配原始数据。非正方形视场角比正方形视场角具有更大的视野范围，因此更具挑战性。微调中的输入视图和新视图均为 64 个，以支持更好的视图覆盖。微调分辨率的课程设置为 128×72 , 256×144 , 512×288 ，以及。每个阶段的训练标记数约为 100B。高分辨率 960×536 模型（从

512×288 ）使用块内上下文并行 (Sec. 3.4) 进行训练。

在每个训练阶段，我们始终使用 AdamW 优化器，并采用线性学习率预热和 0.05 的权重衰减。预训练的峰值学习率为 $4e-4$ 。在微调期间，我们使用较小的学习率（通常为 $1e-5$ 至 $5e-5$ ）。

训练使用 64 块 A100 GPU 完成。预训练耗时 8 天，每个微调阶段约 12 小时（因此总计 2 天）。

³ 原始的 DL3DV 960p 帧以 960×536 分辨率发布。为适应我们建模中的块大小 8，我们将其裁剪为 960×536 ，相机参数也相应调整。

详细结果数值 我们在此提供了 GSO 数据集上物体级结果的详细数值（表 4 中分辨率为 256×256 ，表 5 中为 512×512 ）以及 DL3DV 评估结果（分辨率为 960×536 见表 6）。

表 4: GSO 上 256 分辨率物体级新视角合成结果。输入与输出分辨率均为 256×256 . $\uparrow$: 越高越好, $\downarrow$: 越低越好。

Input Views	# Input Tokens	LaCT			Full Attention			Perceiver Attention		
		PSNR ($\uparrow$)	LPIPS ($\downarrow$)	SSIM ($\uparrow$)	PSNR ($\uparrow$)	LPIPS ($\downarrow$)	SSIM ($\uparrow$)	PSNR ($\uparrow$)	LPIPS ($\downarrow$)	SSIM ($\uparrow$)
4	4,096	32.4	0.030	0.962	32.6	0.029	0.964	30.3	0.039	0.950
8	8,192	35.3	0.019	0.976	35.6	0.018	0.978	32.8	0.026	0.967
12	12,288	36.3	0.017	0.980	36.6	0.015	0.982	33.6	0.023	0.971
20	20,480	37.2	0.015	0.982	37.5	0.014	0.984	34.3	0.021	0.974
32	32,768	37.5	0.014	0.982	37.9	0.013	0.985	34.2	0.021	0.974
48	49,152	37.6	0.014	0.983	37.9	0.013	0.985	33.7	0.022	0.972

表 5: GSO 上 512 分辨率物体级新视角合成结果。输入与输出分辨率均为 512×512 , 跨方法比较。 $\uparrow$: 越高越好, $\downarrow$: 越低越好。

Input Views	# Input Tokens	LaCT			Full Attention		
		PSNR ($\uparrow$)	LPIPS ($\downarrow$)	SSIM ($\uparrow$)	PSNR ($\uparrow$)	LPIPS ($\downarrow$)	SSIM ($\uparrow$)
4	16,384	33.4	0.029	0.969	33.6	0.027	0.971
8	32,768	36.6	0.020	0.979	36.7	0.017	0.982
12	49,152	37.7	0.017	0.983	37.9	0.015	0.985
20	81,920	38.6	0.016	0.984	38.9	0.013	0.987
32	131,072	39.0	0.015	0.985	39.3	0.013	0.988
48	196,608	39.1	0.015	0.985	39.3	0.012	0.988

表 6: LaCT 在 DL3DV 上 960P 场景级新视角合成结果。输入与输出分辨率均为 960×536 (宽 $\times$ 高)。 $\uparrow$: 越高越好, $\downarrow$: 越低越好。

Input Views	# Input Tokens	PSNR ($\uparrow$)	LPIPS ($\downarrow$)	SSIM ($\uparrow$)
16	128,640	24.7	0.224	0.793
32	257,280	26.9	0.185	0.837
64	515,520	28.3	0.169	0.857
128	1,031,520	28.9	0.166	0.861

C.2 语言建模

数据集与度量。我们在长数据集 (Long-Data-Collections) 数据集 [37] 上训练模型, 该数据集包含约 688 亿个标记, 使用 Mixstra 分词器 (码本大小为 32,000) 进行分词。数据集混合了 41.4% 的通用数据 (例如 RedPajama-Book、RedPajama-ArXiv、来自 RedPajama 的 10 亿标记以及 Pile 子样本) 和 58.6% 的指令数据 (例如 UL2 Oscar、NI 和 P3)。为评估长上下文能力, 我们采用了 [38] 中的逐标记损失度量。输入序列上持续下降的逐标记损失表明有效利用了整个上下文, 而平台期则表明无法利用超出该点的信息。具体而言, 我们评估了 760M 参数模型在 Book-3 数据集 [68] 的 25 亿标记上的下一标记预测损失, 以及 3B 参数模型在 50 亿标记上的损失。此外, 我们使用 RULER 基准 [39] 在不同序列长度上测量检索准确率, 以评估上下文记忆和信息检索能力, 评估范围达到训练序列长度。

模型细节。我们修改了原始 LaCT 块, 移除了其窗口注意力层。取而代之, 我们将因果滑动窗口注意力 (SWA) 层直接集成到大块 TTT 层中。SWA 层与快速权重网络共享相同的 Q、K 和 V 向量, 不同之处在于 Q 和 K 在输入 SWA 层之前应用了逐通道可学习的缩放和偏移 (如 GAU[25] 中所做)。我们将 SWA 层的输出与 TTT 层的输出相加, 其中 TTT 层的输出由另一个逐头可学习的标量进行缩放。我们使用快速权重学习率初始化 0.001, 通过将算法 1 中的 ‘const_lr_bias’ 设置为 $\text{softplus}(\text{const_lr_bias}) = 0.001$ 。我们在图 10 中展示了该架构。

为确保在可训练参数方面与基线进行公平比较, 我们调整了拉康块 (LaCT block) 额外的可学习初始快速权重 $W = [W_1, W_2, W_3]$ 。为减少参数, 我们对 W_1, W_3 采用了秩为 32 的低秩版本。例如, 若 $W_1 \in \mathbb{R}^{d \times d}$, 其低秩初始

快速权重为 $W_1 = L \cdot R + 0.5 * I_d$ ，其中 $L \in \mathcal{R}^{d \times 32}$, $R \in \mathcal{R}^{32 \times d}$ ，且 I_d 为单位矩阵。这将每个块中快速权重的额外可训练参数减少至 $128 * \text{模型维度} + \frac{1}{\text{num-heads}} (\text{模型维度}^2)$ 。用于学习率投影、每头缩放器、额外均方根归一化（RMSNorm）以及滑动窗口注意力（SWA）的可学习缩放与偏移的少量额外参数为 $O(\text{模型维度})$ 量级。标准块通常具有 $12 * \text{模型维度}^2$ 个参数。我们的方法额外增加约 $\frac{1}{\text{num-heads}} (\text{模型维度}^2)$ 个参数，在四个头时不到总可训练权重的 3%，在两个头时低于 4.5%。除非另有说明，实验中拉康块默认使用四个头，这意味着每个块的默认状态大小为 $\frac{3}{4} (\text{模型维度}^2)$ 。

基线。 我们将所提方法与全注意力、门控线性注意力（Gated Linear Attention, GLA）[13]、德尔塔网络（DeltaNet）[3, 15] 进行比较。为确保公平，我们为 GLA 和德尔塔网络均增强了相同的滑动窗口注意力。如先前工作 [38, 40, 41] 所指出的，较大的旋转位置编码（RoPE）[42] 基数对于变换器在长上下文训练中至关重要，因此在使用 Softmax 注意力时，我们采用 100 万的较大旋转位置编码基数进行 32K 词元上下文的训练。表 7 比较了基线方法与本方法的机制和计算复杂度。训练吞吐量（每 GPU 每秒词元数，TPS）使用一个 30 亿参数模型在八块 A100-40GB SXM4 GPU 上测量，并采用激活检查点和全分片数据并行（FSDP）。在 30 亿参数规模下，所有模型均使用 24 个 Softmax 注意力头。GLA 基线具有八个线性注意力头，头维度为 384，总状态大小为 $384d$ ，其中 $d = 3072$ 表示模型维度。德尔塔网络采用 24 个线性注意力头，每个头维度为 128，总状态大小为 $128d$ 。我们的方法使用四个测试时训练（TTT）头，头维度为 768，且由于每个块具有三个快速权重，总状态大小为 $2304d$ 。

表 7: 基线方法在状态大小、训练吞吐量（以每秒词元数 TPS 衡量）、更新规则和内存读出机制方面的比较。训练吞吐量使用一个 30 亿参数模型在 A100-40GB GPU 上以 32K 序列长度进行评估。

	State size	Train TPS	Update Rule	Memory read-out
Transformer	–	4.1K	–	–
Transformer SWA	–	6.4K	–	–
<i>Per-token recurrence</i>				
GLA SWA	$384d$	5.0K	$\mathbf{S}_t \leftarrow \mathbf{S}_{t-1} \text{Diag}(\alpha_t) + \mathbf{v}_t \mathbf{k}_t^\top$	$\mathbf{o}_t = \mathbf{S}_t \mathbf{q}_t$
DeltaNet SWA	$128d$	5.1K	$\mathbf{S}_t \leftarrow \mathbf{S}_{t-1} (\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top) + \beta_t \mathbf{v}_t \mathbf{k}_t^\top$	$\mathbf{o}_t = \mathbf{S}_t \mathbf{q}_t$
<i>Large-chunk recurrence</i>				
Ours GD	$2304d$	5.0K	$W \leftarrow \text{L2norm}(W - \sum_i \eta_i \nabla_W \mathcal{L}_i)$	$\mathbf{o}_t = f_W(\mathbf{q}_t)$
Ours Momentum	$2304d$	4.9K	$M \leftarrow \beta M + \sum_i \eta_i \nabla_W \mathcal{L}_i$; $W \leftarrow \text{L2norm}(W - M)$	$\mathbf{o}_t = f_W(\mathbf{q}_t)$
Ours Muon	$2304d$	4.3K	$M \leftarrow \beta M + \sum_i \eta_i \nabla_W \mathcal{L}_i$; $W \leftarrow \text{L2norm}(W - \text{Muon}(M))$	$\mathbf{o}_t = f_W(\mathbf{q}_t)$

训练细节。 我们使用 32,768 个词元的序列长度在两种规模下训练模型：

- 7.6 亿参数：采用 24 个堆叠块，模型维度为 1536。所有模型均训练 40B 个词元（40,960 步），滑动窗口为 2048 个词元，批大小为 100 万个词元。每个实验在 32 块 A100-40GB SXM GPU 上运行约 20 小时。
- 30 亿参数：采用 25 个堆叠块，模型维度为 3072。所有模型均训练 60B 个词元（30,000 步），滑动窗口为 4096 个词元，批大小为 200 万个词元。每个实验在 64 块 A100-40GB SXM GPU 上运行约 50-60 小时。

对于两种规模，均使用基础学习率 1×10^{-3} ，并采用余弦衰减调度器和 1024 个预热步骤。所有模型均以标准差 0.02 进行随机初始化。

结果。 在 RULER 基准 [39] 上的详细结果见表 8 和表 9。我们在 S-NIAH-1、S-NIAH-2 和 S-NIAH-3 任务上评估了模型，这些任务代表了单一“大海捞针”检索的不同难度。我们还报告了 NIAH-MultiKey-1、NIAH-MultiQuery 和 NIAH-MultiValue 上的性能。其他 RULER 任务未报告，因为全注意力基线在超过 16K 序列长度时也取得了微不足道的结果。

表 8: RULER 基准上单针大海捞针 (S-NIAH) 任务的结果。* 我们的方法使用两个头（默认为四个）。

Model	S-NIAH-1				S-NIAH-2				S-NIAH-3				Average			
	4K	8K	16K	32K	4K	8K	16K	32K	4K	8K	16K	32K	4K	8K	16K	32K
<i>760M parameters</i>																
Transformer	99.2	96.6	85.2	68.0	100	100	85.8	82.2	81.0	73.8	74.8	36.8	93.4	90.1	81.9	62.3
DeltaNet + SWA	84.0	85.2	87.8	86.8	62.8	29.4	14.2	7.8	53.8	21.8	11.2	5.8	66.9	45.5	37.7	33.5
GLA + SWA	51.8	26.2	14.4	8.6	55.8	26.4	15.8	7.8	58.0	23.8	16.2	5.0	55.2	25.5	15.5	7.1
Ours	94.8	53.2	26.0	14.8	74.0	28.0	14.2	7.8	42.8	26.6	14.4	6.8	70.5	35.9	18.2	9.8
Ours Momentum	95.6	84.8	83.4	84.8	91.4	73.4	22.8	7.8	82.6	34.8	16.6	6.6	89.9	64.3	40.9	33.1
Ours Momentum*	59.0	30.0	12.4	8.4	93.4	50.0	18.2	7.8	60.2	25.6	14.2	6.8	70.9	35.2	14.9	7.7
Ours Muon	98.0	95.0	92.2	92.4	86.6	60.2	17.0	7.8	49.2	26.2	10.9	5.2	77.9	60.5	40.0	35.1
<i>3B parameters</i>																
Transformer	100	100	100	100	100	99.8	100	98.6	98.6	95.8	90.8	75.0	99.5	98.5	96.9	91.2
GLA SWA	100	52.8	26.0	13.2	100	51.8	29.6	14.4	98.0	54.4	27.6	12.4	99.3	53.0	27.7	13.3
DeltaNet SWA	100	89.6	76.2	54.8	100	76.4	42.2	17.0	90.6	57.6	27.4	13.4	96.9	74.5	48.6	28.4
Ours Momentum	99.4	97.0	98.6	93.4	100	75.6	39.6	15.0	91.8	63.0	27.8	13.4	97.1	78.5	55.3	40.6
Ours Muon	98.8	99.2	98.6	93.4	100	99.0	83.2	30.8	95.4	90.8	55.6	19.8	98.1	96.3	79.1	48.0

表 9: RULER 基准上多键 (MK-NIAH)、多查询 (MQ-NIAH) 和多值 (MV-NIAH) 大海捞针任务的性能。* 我们的方法使用两个头（默认为四个）。

Model	MK-NIAH				MQ-NIAH				MV-NIAH				Average			
	4K	8K	16K	32K	4K	8K	16K	32K	4K	8K	16K	32K	4K	8K	16K	32K
<i>760M parameters</i>																
Transformer	63.8	72	71.4	54	33.4	28.9	24	23.1	27.95	24	20.5	27.35	41.7	41.6	38.6	34.8
DeltaNet+SWA	41.2	30	14.6	8.2	33	22.45	7.5	4.3	32.4	22.8	9.15	6.6	35.5	25.1	10.4	6.4
GLA + SWA	45.4	28.4	15.8	6.6	26.1	17.75	10.2	5.85	25.4	16.85	10.1	6.6	32.3	21.0	12.0	6.3
Ours	60.8	34.6	16.8	7	35	23.65	14.1	7.45	20.7	22.05	12.7	6.85	38.8	26.8	14.5	7.1
Ours Momentum	62	41	21.2	10.4	35.3	24.95	17.7	8.6	27.9	23.15	16.65	8.2	41.7	29.7	18.5	9.1
Ours Momentum*	59.8	37.8	19.2	8.8	36.65	20.45	12.5	7.4	24.45	16.95	11.6	6.8	40.3	25.1	14.4	7.7
Ours Muon	62.8	46.6	22	8.6	37.7	26.55	15.7	7.1	28.35	23.15	13.6	6.85	42.9	32.1	17.1	7.5
<i>3B parameters</i>																
Transformer	95	90.4	81.6	65.2	86.45	81.55	71.70	40.85	61.8	42.8	30.75	22.9	81.1	71.6	61.4	43.0
GLA 3B	78	45.8	28.6	14.4	50.05	28.05	19	10.7	29.4	21.4	16.75	9.9	52.5	31.8	21.4	11.7
DeltaNet SWA	75.8	57.4	34.2	17.8	66.25	33.05	21.45	13.45	43.7	23.2	18.85	13.2	61.9	37.9	24.8	14.8
Ours Momentum	96.2	59.6	35	17.2	87.05	40.25	25.6	13.2	88.08	30.65	21.9	12.3	90.4	43.5	27.5	14.2
Ours Muon	75.2	69.2	46.2	25.2	44.75	39.1	24.9	19	26.55	29.1	25.05	19.3	48.8	45.8	32.0	21.2

C.3 自回归视频扩散

我们将预训练的 Wan 2.1[43] 文本到视频扩散模型微调为自回归视频扩散模型，该模型通过迭代去噪连续的视频帧块来生成视频。

模型细节。原始的 Wan 2.1 是一种双向扩散 Transformer，作用于因果视频变分自编码器 (VAE) 的潜空间，该 VAE 执行 8 倍空间和 4 倍时间下采样。扩散 Transformer 使用 $2 \times 2 \times 1$ 分块化层将 VAE 视频潜变量转换为令牌。扩散 Transformer 的每个块包含一个多层感知机 (MLP) 层、一个用于视觉令牌的双向自注意力层，以及一个用于视觉和文本令牌的交叉注意力层。

我们的主要修改针对双向自注意力。我们首先将其替换为块因果滑动窗口注意力 (SWA)，窗口大小为两个视频帧块。然后，我们在同一层中集成我们的 LaCT。我们为 LaCT 初始化可学习的快速权重。与我们的语言建模实验一致，SWA 和我们的测试时训练机制在每一层内结合：Q 和 K 向量在输入测试时训练操作之前进行缩放和偏移。SWA 和测试时训练层的输出相加，后者应用每个头的可学习标量（来自零初始化的线性投影）。我们在快速权重更新中不使用 Muon，因为经验上它在验证损失上没有显著差异。我们使用快速权重学习率初始化 0.001，通过将算法 1 中的 ‘const_lr_bias’ 设置为 $\text{softplus}(\text{const_lr_bias}) = 0.001$ 。这允许在微调开始时对快速权重进行小幅更新。为了保持对原始 Wan 架构的最小更改，LaCT 层利用 Wan 模型中的原始旋转位置编码 (RoPE)，并且我们移除了先前应用于查询和值的 SiLU 激活函数。

数据集。我们使用内部过滤的专有视频集合对模型进行微调，每个视频都配有由视觉语言模型生成的简短文本提示 [69]。

训练细节。遵循 [44, 43], 我们使用时间步偏移 (缩放因子 3.0) 和对数正态去噪损失加权 (mean=0.5, std=1.0)。我们还对模型权重应用衰减率为 0.995 的指数移动平均。每个 5 秒视频 (16 FPS, 480×832 分辨率) 由 Wan VAE 编码为 [21,60,104] 潜表示。去噪以三个潜帧 (每个 4680 个视觉令牌) 的块自回归执行。我们采用教师强制, 使用交错的有噪-无噪块序列 (见第 4.3 节)。

- **1.3 亿参数模型:** 对于 5 秒视频的初始训练, 噪声块重复两次。这导致序列包含 60 个潜在帧 (14 个噪声块, 6 个干净块), 总计 93,600 个标记。我们以批大小 64 对模型进行 5000 次迭代的微调。基础学习率设置为 2×10^{-5} , 采用 1000 次迭代的线性预热和线性衰减。随后, 模型在 10 秒视频片段上微调 1000 次迭代。这些片段对应于干净视频部分的 42 个潜在帧, 形成 81 个潜在帧的交错序列 (包括噪声块约 126K 个标记)。5 秒视频的训练在 64 块 A100 80GB SXM GPU 上每次迭代耗时 ~ 20 秒 (或在 64 块 H100 80GB SXM GPU 上耗时 ~ 10 秒)。

- **1.4B 参数模型:** 为了管理 GPU 内存使用, 此设置中不重复噪声块。我们以批大小 64 对 5 秒视频训练 5000 次迭代, 基础学习率为 5×10^{-6} , 并使用序列并行大小为 2 块 GPU。此阶段在 64 块 A100 GPU 上每次迭代耗时 ~ 80 秒。随后, 模型在 8.8 秒视频片段 (干净部分 36 个潜在帧) 上额外微调 600 次迭代, 使用序列并行 (4 块 GPU)。此微调在 64 块 H100 GPU 上每次迭代耗时 ~ 80 秒。

基线。我们将我们的方法与三种基线进行比较: 滑动窗口注意力、带滑动窗口注意力的 Mamba2[23], 以及全块级因果注意力, 其中基线中的窗口注意力实现方式与我们的模型相同。对于 Mamba2 层, 我们遵循 [70], 将原始投影 k, q, v 分别作为 B, C 和 x 应用。Mamba2 的状态逐标记更新, 我们在处理噪声帧块后回退状态, 以确保只有干净块的状态更新得以传播。全块级因果注意力基线使用 FlexAttention[71] 实现。

评估。我们在 5000 次训练迭代后, 对 2000 个视频的集合验证所有模型, 通过在五个时间步 (550、650、750、850、950) 计算去噪损失。去噪损失针对每个视频帧块进行测量, 并绘制在图 6 中。图 6(a) 比较了 LaCT 与 SWA、带 SWA 的 Mamba2 以及全块级因果注意力的验证损失 (最长 5 秒视频)。我们的 LaCT 与全注意力相当, 并优于其他基线。图 6(b) 展示了使用不同窗口大小的 SWA 基线与我们的方法及基线的比较 (最长 5 秒视频)。默认窗口覆盖六个潜在帧 (两个块)。额外实验使用了四帧窗口。结果表明, 将窗口大小从四帧增加到六帧可改善验证损失, 但这种改善小于引入 LaCT 所获得的改善。图 6(c) 展示了在 10 秒视频上微调 LaCT 和 SWA 基线 1000 次迭代后的验证损失 (最长 10 秒视频)。

我们模型的生成视频样本在附加文件夹中提供。每个视频块按照原始 Wan 方法采样, 使用 UniPC [72] 采样器, 50 步, 无分类器引导 5.0, 时间步偏移 3.0。

C.4 图 1 中的实验细节

图 1(c) 展示了训练一个 7.6 亿参数 LaCT 语言模型的结果。我们采用 SwiGLU MLP 快速权重和 Muon 测试时优化器。为了扩展快速权重的大小, 我们将快速权重 MLP 的中间维度固定为与头维度匹配, 然后将头维度从 128 增加到 1536, 同时按比例减少头数, 以保持总模型维度不变。验证损失在每个 32768 个标记序列的最后 2048 个标记上计算, 并在 Book3 数据集的 76K 个序列上取平均。

图 1(d) 使用对象级新视角合成实验。所有模型由 14 个堆叠块组成, 固定模型维度为 768, 训练了 1670 亿个标记。训练时间 (墙钟时间) 在 A100-40GB SXM GPU 上测量。

D Mamba 基线的详细信息

Mamba 是一种高效的模型架构，其逻辑上类似于线性 TTT，采用逐令牌的快速权重（即 Mamba 语境中的状态）线性更新规则。因此，它可作为基线，用于理解图 8 和图 6 中分块更新与逐令牌更新之间的差距。本节详细介绍了实验设置。在所有实验中，我们均采用官方 Mamba-2 实现⁴。原始 Mamba-2 包含多个组件，我们大幅简化其实现，以保持架构可度量性，同时维持性能。具体而言，实验中 Mamba-2 的公式为： $X, B, C, \delta = \text{Linear}(u) = \text{softplus}(\delta + \delta_{init})$

$$\begin{aligned} \delta &= \delta + \delta_{init} \\ H_t &= \exp(-\delta_t) H_{t-1} + \delta_t B_t^T X_t \\ y_t &= C_t H_t \end{aligned} \quad (23)$$

其中 X, B, C 的形状为 (L, d) ， δ 的形状为 $(L, 1)$ 。 H_t ，是一个形状为 (d, d) 的矩阵状态。将上述公式转换为标准的线性注意力/TTT/DeltaNet 表示法，其等价于： $= \text{Linear}(\text{input})$

$$\begin{aligned} V, K, Q, l_r &= \text{Linear}(\text{input}) \\ l_r &= \text{softplus}(l_r + l_{r_{init}}) \\ W_t &= \exp(-l_r) W_{t-1} + l_r K_t^T V_t \\ O_t &= Q_t W_t \end{aligned} \quad (24)$$

我们将上述方程记为 $O = \text{Mamba}(\text{input})$ 。

我们采用与 Transformer 多头注意力类似的多头设计。多个独立的 Mamba-2 层并行运行，其输出被拼接。假设头数为 nh ，公式为： $O^k = \text{Mamba}^k(\text{input})$

$$O = [O^1, \dots, O^{nh}] \quad (25)$$

其中每个 Mamba^k 是一个具有自身参数的 Mamba。在 Mamba-2 的术语中，此设计等价于将“组”数设置为与头数相同。

对于新视角合成任务，我们对输入图像令牌采用双向 Mamba。具体而言，我们使用两个独立的多头 Mamba，一个从左到右读取，另一个从右到左读取。双向模型在输入令牌之间建立了更好的连接，同时使状态大小加倍。我们采用与 LaCT 类似的“应用”操作，仅对输入令牌更新状态，而目标令牌的状态保持静态。我们也测试了对目标图像令牌进行“更新”，但经验表明这会导致更差的结果。我们使用 192 的头维度和 8 个头。总状态大小为 $8 \times 192^2 \times 2$ （双向），与 LaCT 中维度为 768（768 输入维度 $\times$ 768 中间维度）的标准大块大权重线性注意力相匹配，如图 8 所示。我们取 $l_{r_{init}} = -4.6$ ，其对应于 0.01 初始化的学习率（即 $\text{softplus}(-4.6) = 0.01$ ）。

对于自回归视频扩散任务，我们在展平的视频标记上应用单向曼巴（Mamba）。如第 5.3 节所述，我们遵循 [70] 继承 Wan 的自注意力投影 k, q ，并将 v 作为 B, C ，将 x 分别用于曼巴层。与 NVS 任务不同，每个标记都会“更新”状态，该状态将“应用”于当前输出和未来标记。我们的曼巴使用 12 个头，每个头维度为 128，与 Wan 中原始的多头自注意力相匹配。总体状态大小为 12×128^2 （头维度）。8*19 我们取 $l_{r_{init}} = -4.6$ ，这对应于 0.01 初始化的学习率。

E LaCT 上下文并行实现细节

上下文并行（Context Parallelism, CP）沿序列长度维度划分输入序列，并将分片分布到多个设备上并行计算。前馈层和 ⁴ <https://github.com/state-spaces/mamba>

Algorithm 3 Large Chunk Test-Time Training Layer with Context Parallel Sharded inside chunk Pseudocode

```
def update(fast_weight, k, v, lr, cp_group, use_muon=True):
    """
    Fast-weight update for a SwiGLU MLP using a context-parallel chunk.

    Args:
        fast_weight : tuple(w1, w2, w3) with shapes: w1, w3: [b, d, dh]; w2: [b, dh, d]
        k, v : key / value tensor of shape [b, l, d]
        lr : per-token learning rates of shape [b, l, 3] -> (lr1, lr2, lr3)
        cp_group : process group metadata for context parallelism
        use_muon : whether to apply Muon to orthogonalize the update

    Note:
        The input tensors k, v, lr are assumed to be already partitioned (sharded) along the sequence
        dimension over multiple devices. l represents the local sharded sequence length on each device.
        The total effective chunk size processed is l * cp_group.size.
    """

    # Forward with k:
    gate_before_act = matmul(k, w1) # [b, l, dh] = [b, l, d] x [b, d, dh]
    hidden_before_gate = matmul(k, w3) # [b, l, dh] = [b, l, d] x [b, d, dh]
    hidden = silu(gate_before_act) * hidden_before_gate

    # Backward:
    dhidden = matmul(v, w2.transpose(-1, -2)) # [b, l, dh] = [b, l, d] x [b, dh, d]
    dhidden_before_gate = dhidden * silu(gate_before_act)
    dgate = dhidden * hidden_before_gate
    dgate_before_act = silu_backprop(dgate, gate_before_act)

    # Compute gradients:
    w2.grad = -matmul(hidden.transpose(-1, -2), v * lr2) # [b, dh, d] = [b, dh, l] x [b, l, d]
    # [b, d, dh] = [b, d, l] x [b, l, dh]
    w1.grad = -matmul((k * lr1).transpose(-1, -2), dgate_before_act)
    w3.grad = -matmul((k * lr3).transpose(-1, -2), dhidden_before_gate)

    # [Standard forward pass and local backward gradient computations are performed above,
    # resulting in local w.grad for each device.]

    #####
    # BEGIN CONTEXT PARALLELISM SPECIFIC MODIFICATION: Global Gradient Aggregation
    # The following AllReduce operation is the key step introduced for context
    # parallelism. Operations before this point compute local gradients; operations
    # after this point use the globally aggregated gradients.
    #####
    for w in fast_weight:
        w.grad = distributed_all_reduce(w.grad, cp_group, op="SUM")
    #####
    # END CONTEXT PARALLELISM SPECIFIC MODIFICATION.
    # Subsequent operations (Muon, weight updates) now use the globally summed w.grad.
    # The formulas for these subsequent operations remain the same as in a
    # non-parallel version, but they act upon these aggregated gradients.
    #####

    # Weight update
    if use_muon:
        for w in fast_weight:
            w.grad = zeropower_via_newtonschulz5(w.grad)
    for w in fast_weight:
        w = (w - w.grad) / (w - w.grad).norm(dim=1) * w.norm(dim=1)

    return fast_weight
```

窗口注意力是局部操作，因此天然支持 CP。我们的大块测试时训练 (TTT) 方法通过在每个大块内分片标记来促进 CP。

在我们的大块 TTT 机制中，逐标记应用操作由于其独立性而天然支持 CP。更新操作通过将块内标记分片到多个设备上支持 CP。这种 CP 可以通过在每个设备上计算局部快速权重梯度后添加几行分布式全归约求和来轻松实现，逻辑上与分布式数据并行相同。注意，分布式全归约求和是可微操作，其反向传播是对梯度的全归约求和，因此网络可以端到端训练。算法 3 展示了专门针对大块 TTT 更新操作的块内上下文并行的伪代码。我们在视图合成实验中采用了这种并行方式，训练期间处理的最大块大小超过五十万个标记，最大序列长度超过一百万个标记。

F LaCT 张量并行实现细节

除了上下文并行，我们的大块测试时训练（TTT）机制还支持张量并行（Tensor Parallelism, TP）。这主要通过将 TTT 头分片到多个设备上来实现，类似于 DeepSpeed Ulysses[73] 等方法中采用的策略。

具体而言，虽然模型中的静态前馈层可能处理沿序列维度切分的输入（上下文并行），但对于我们的拉普拉斯变换层中的测试时训练操作，数据经历先收集后分散的变换。初始按序列长度切分的输入张量（查询、键、值以及测试时训练的学习率）首先沿序列维度收集，以在张量并行组内的每个设备上重建完整的序列上下文。然后，这些完整序列张量沿头维度分散。因此，每个设备处理完整序列，但在测试时训练更新和应用迭代期间仅操作其分配到的测试时训练头子集。反向变换（收集头，分散序列）应用于测试时训练操作的输出。算法 4 提供了伪代码，详细说明了这种张量并行实现，省略了填充等次要细节。虽然这种先收集后分散的方法有效地实现了按头切分的张量并行，但更复杂的通信策略 [74, 73] 可能被用来进一步优化通信开销。

我们在自回归视频生成实验中采用了这种张量并行策略，例如，将四个测试时训练头切分到四个本地图形处理器上。这使我们能够训练具有超过 10 万标记序列长度的 140 亿参数扩散模型。

Algorithm 4 Large Chunk Test-Time Training Layer with Tensor Parallelism by sharding heads
Pseudocode

```
def gather_scatter(x, gather_dim, scatter_dim, process_group=cp_group):
    """
    Gathers tensor x along gather_dim across process_group,
    then scatters the result along scatter_dim to each device locally.
    Example: Transform [B, N_full, L_local, D] with gather_dim=2, scatter_dim=1
             to [B, N_local, L_full, D] on each device.
    """
    x = all_gather(x, gather_dim, process_group)

    # Calculate slicing indices for the scatter operation
    local_rank, group_size = process_group.rank, process_group.size
    scatter_stride = x.size(scatter_dim) // group_size
    start_idx = local_rank * scatter_stride
    end_idx = (local_rank + 1) * scatter_stride

    # Slice the tensor to get the local shard for the current device
    x = slice_tensor(x, scatter_dim, start_idx, end_idx)
    return x

##### MultiHead LaCT Layer with Tensor Parallelism (sharding TTT heads) #####
# Input:
# x: input sequence sharded by sequence length (CP). Shape [b, l, d], b is the batch dim, l is local
#     sequence length, d is model dimension.
# fast_weight: tuple of sharded initial fast weights, shared among heads. (w1, w2, w3); w1, w3 of
#     shape [nh, d, dh], w2 of shape [nh, dh, d]. nh: number of local heads.

qkv = silu(LinearQKV(x)) # [b, l, d * 3]
qkv = rearrange(qkv, 'b l (nh hd) -> b nh l hd', nh=num_heads).split(3, dim=-1)
q, k = q / q.norm(-1), k / k.norm(-1)
lr = softplus(LinearLR(x) + const_lr_bias) # [b, l, 3 * num_heads]
lr = rearrange(lr, 'b l (nh 3) -> b nh l 3', nh=num_heads)

#####
# BEGIN TENSOR PARALLELISM SPECIFIC TRANSFORMATION
# Gather along Sequence Length (dim 2), then Scatter along Head Dimension (dim 1).
# Transforms [b, nh_full, l_local, X] -> [b, nh_local, l_full, X]
# Each device now has the full sequence for a subset of heads.
q, k, v, lr = map(lambda x: gather_scatter(x, gather_dim=2, scatter_dim=1), (q, k, v, lr))
# END TENSOR PARALLELISM SPECIFIC TRANSFORMATION
#####

# [b, nh_local, l_full, X]
o_local_heads = ... # Placeholder for actual TTT computation on sharded heads
o_local_heads = RMSNorm(o_local_heads) # per-head norm

#####
# BEGIN TENSOR PARALLELISM SPECIFIC REVERSE TRANSFORMATION
# Gather along Head Dimension (dim 1), then Scatter along Sequence Dimension (dim 2).
# Transforms [b, nh_local, l_full, X] -> [b, nh_full, l_local, X]
# This reconstructs the full head dimension but shards sequence back.
o = gather_scatter(o_local_heads, gather_dim=1, scatter_dim=2)
# END TENSOR PARALLELISM SPECIFIC REVERSE TRANSFORMATION
#####
o = rearrange(o, 'b nh l hd -> b l (nh hd)', nh=num_heads)
o = LinearOutput(o)

return o
```
